

Внутрішні налаштування інтернет-магазину Termojet

Технічний посібник для побудови нового магазину обладнання для систем опалення

Джерело: робочий код і продакшн-конфігурація сайту **termojet.com.ua** станом на 17 серпня 2026 року.
Обсяг: адмінка, база даних, API, SEO-механіка, копірайтинг, структура блоків сторінок, мультимовність, комерція, інтеграції, безпека, швидкодія, деплой. **Свідомо поза обсягом:** тема дизайну (кольори, шрифти, компонування, анімації) — новий магазин отримає власне візуальне рішення.

Зміст

- Технічний посібник для побудови нового магазину обладнання для систем опалення
- Зміст
- 0. Як читати цей документ
- 1. Архітектура і принципові рішення
- 2. База даних: повна схема
- 3. API: повний перелік і правила
- 4. Адмінка: розділи, поля, реальні межі
- 5. Каталог: структура і таксономія
- 6. SEO: повна механіка
- 7. Копірайтинг: методика, за якою написано 331 опис
- 8. Структура блоків сторінок
- 9. Мультимовність
- 10. Комерція: ціни, кошик, замовлення, доставка
- 11. Інтеграції: повний реєстр
- 12. Безпека
- 13. Продуктивність
- 14. Деплой і оточення
- 15. Дані: імпорт, синхронізація, прайс
- 16. Чеклісти
- 17. Зведений реєстр граблів
- Додаток А. Приклад JSON-LD сторінки товару
- Додаток В. Шаблон ТЗ копірайтеру
- Додаток С. Команди щоденної експлуатації
- Підсумок: сім рішень, які дали найбільше

0. Як читати цей документ

Документ описує **внутрішню начинку** працюючого магазину: 331 товар, 15 категорій, 5 мов, повний цикл лід → CRM → Telegram → лист клієнту. Усе, що тут написано, вже пройшло продакшн і кілька ре-аудитів.

Актуальна оцінка squirrelscan — 77/100 (grade C), повний скан 500 сторінок 17 серпня 2026. Максимум за час проєкту був **81 (B)** 10 липня. Падіння на 4 бали за п'ять тижнів — не збій коду, а **операційні наслідки заливки нових товарів** (розділ 6.16). Це найважливіший практичний урок усього документа: **технічне SEO магазину деградує само, якщо процес наповнення не має захисних кроків**.

Кожен розділ побудовано за однією схемою:

Позначка	Значення
Як зроблено	фактичний стан коду termojet.com.ua
⚠ Граблі	реальна помилка, на яку ми вже наступили — не повторювати
✅ Правило	те, що треба закласти в новий магазин з першого дня
💡 Зробити краще	місце, де наше рішення вимушене й у новому проєкті варто інакше

Головна теза документа. У магазині обладнання для опалення 80 % результату дають не функції, а **дисципліна даних**: одна таблиця товарів як джерело істини, обов'язкові SEO-поля на кожній позиції, характеристики у структурованому вигляді, і механізм, який віддає все це краулеру в сирому HTML. Решта — обгортка.

1. Архітектура і принципові рішення

1.1 Стек

Шар	Технологія	Роль
Фронтенд	React 19 + Vite, SPA	усі публічні сторінки й адмінка
Роутинг	react-router-dom (BrowserRouter)	24 публічних роути × 5 мов + 20 адмінських
Бекенд	Node.js + Express	API, серверний SEO-ін'єкт, роздача білду
БД	SQLite через <code>better-sqlite3</code> (WAL)	15 таблиць, файл у Docker-volume
Стилі	Tailwind + CSS-змінні	(поза обсягом документа)
Хостинг	Hetzner VPS, Docker Compose	один контейнер: Express роздає <code>dist/</code> + <code>/api</code>
Периметр	nginx + Let's Encrypt	TLS, 301 <code>www</code> →без- <code>www</code> , дзеркало старих фото
Аналітика	Google Tag Manager + GA4	ecommerce-події через <code>dataLayer</code>

Чому SQLite, а не PostgreSQL. Магазин на 331 товар з десятками замовлень на тиждень — це навантаження, для якого SQLite у режимі WAL надлишковий із запасом. Плюси, які виявились критичними на практиці: БД — це **один файл**, тому бекап робиться копіюванням, а перенесення між серверами — `scp`. Мінус, який теж з'явився: файл лежить у volume, і **його вмісту немає в git** (розділ 15).

✅ **Правило.** Для магазину до ~5000 SKU і без складських залишків у реальному часі SQLite достатній. Переходити на Postgres варто тоді, коли з'явиться одночасний запис із кількох процесів (синхронізація з 1С + адмінка + сайт).

1.2 Головне архітектурне рішення: як SPA віддає SEO

Це найважливіший розділ усього документа, бо саме тут ми витратили найбільше часу й пройшли три невдалі підходи.

Проблема. React SPA віддає краулеру порожню оболонку: `index.html` без контенту. Google з часом виконує JS і бачить сторінку, але: - перша хвиля краулінгу індексує generic-метадані з оболонки; - **LLM-боти (ChatGPT, Perplexity, Gemini, Copilot) JS не виконують взагалі** — на старті ми мали 0 згадок у всіх чотирьох при живих 4 згадках в AI Overviews Google.

Аудит фіксував на 361 сторінці: canonical на всіх сторінках вів на головну, 269 задовгих title, 361 дубль-description, 349 «orphan»-сторінок.

Три підходи, які ми відкинули (з причинами):

Підхід	Що зробили	Чому не пішло
Клієнтський <code>react-helmet-async</code>	ставив правильні теги після монтування React	дублював серверні теги (у DOM 2 копії title/description); краулер без JS не бачив нічого
Prerender (Puppeteer → <code>dist/<route>/index.html</code>)	спайк <code>hydrateRoot</code>	hydration mismatch React error #418 : helmet мутує <code><head></code> , у знімку теги подвоюються

Підхід	Що зробили	Чому не пішло
Prerender після міграції на нативні метадані React 19	mismatch зник	нативні метадані React 19 — runtime-only: у DOM-знімку їх немає, тобто краулеру без JS вони не дістаються. DOM-snapshot prerender як ідея не годиться

Обране рішення (внутрішня назва «А+»): серверний ін'єкт метаданих і контенту з БД.

Express перехоплює URL сторінок, читає готові поля з SQLite і **вписує їх у сирий HTML** перед відправкою. React на клієнті рендерить свою версію як завжди.

```
GET /catalog/nasosni-hrupy/ng-46p-z-termokranom-1
|
├ Express: SELECT ... FROM products WHERE slug=? AND is_visible=1
├ читає dist/index.html як текст
├ injectMeta(): title ← seo_title, canonical=self, description ← meta_description,
|               og:*, twitter:*, hreflang uk/en/x-default, JSON-LD
├ #seo-content ← повний семантичний HTML товару (H1, фото, ціна,
|               наявність, опис, таблиця характеристик, схожі товари, навігація)
└ віддає HTML → краулер бачить усе одразу, React монтується поверх
```

Результат за метриками `squirreldscan` (три зрізи: старт → пік → сьогодні):

Метрика	8 лип (старт)	10 лип (пік)	17 сер (зараз)
Загальна оцінка	73 (C)	81 (B) , failed 5→0	77 (C) , failed 11
Core SEO	90	99	94
Content	82	93	84
Performance	90	94	85
Images	100	100	86
Structured Data / Local SEO / Social / Mobile / i18n / Analytics	—	100	100
Accessibility	93	93	94
Crawlability · URL	98 · 98	98 · 98	98 · 98
E-E-A-T	61	68	68
Legal	44	44	44
<code>meta-title</code> > 60 символів	269	0	0
<code>duplicate-description</code>	361	30 → 0	0
<code>og:url</code> ≠ canonical	331	0	0

Головні досягнення механіки ін'єкту (title, canonical, og, структуровані дані) **тримаються** — вони закладені в код. Просіли рівно ті метрики, що залежать від **дисципліни наповнення**: Content, Images, Performance і повернені дублі title. Розбір — у розділі 6.16.

💡 **Зробити краще.** Ін'єкт — це вимушений обхід для наявного CSR-застосунку. Якщо новий магазин пишеться **з нуля**, беріть фреймворк із серверним рендерингом (Next.js App Router, Remix, Astro): усе з розділу 6 отримаєте «з коробки», без ручного дублювання логіки метаданих між фронтом і бекендом. Але якщо стек уже обрано як CSR SPA — механіка з розділу 6 доведена і працює.

1.3 Структура репозиторію

```
website/
├─ index.html          ← оболонка з плейсхолдерами для SEO-ін'єкту
├─ package.json        ← build:prod = sitemap + VITE_BASE_URL=/ vite build
├─ Dockerfile / docker-compose.yml
├─ .env.example        ← реєстр усіх ENV-змінних (значення – на сервері)
├─ src/                ← React
│   ├─ App.jsx         ← роутинг: UA + 4 мовні дерева + адмінка
│   ├─ pages/          ← 24 публічні сторінки
│   │   └─ admin/      ← 20 адмінських сторінок
│   ├─ components/     ← спільні компоненти (SEO.jsx, галерея, 3D-в'юер...)
│   └─ data/           ← categories.js, files.js, docsMapping.js + статичний фолбек
товарів
├─ i18n/               ← translations.js (усі рядки інтерфейсу 5 мовами)
├─ utils/              ← currency, analytics, utm, slug, localizedPath, localizeHtml
├─ context/AppContext.jsx ← стан: товари, кошик, мова, адмін-токен
├─ backend/
│   ├─ server.js        ← Express + УБЕСь SEO-ін'єкт (907 рядків)
│   ├─ db/index.js      ← схема, міграції, сідинг (410 рядків)
│   ├─ routes/          ← 23 роутери API
│   ├─ telegram.js crm.js email.js currency.js
│   ├─ redirects.json   ← 280 правил 301 зі старих URL
│   ├─ seed-products.json ← початковий каталог
│   ├─ scripts/         ← apply-*.js: масові правки даних
│   ├─ data/termojet.db ← ⚠ volume, НЕ в git
│   └─ uploads/         ← ⚠ volume, НЕ в git
├─ scripts/            ← gen-sitemap, seo-fix-titles, gen-redirects, i18n-merge, seed-faqs
├─ public/             ← robots.txt, sitemap.xml, фото, PDF-документація
└─ docs/               ← SEO-PLAN.md, DEPLOY.md, GSC-instrukciya.md
```

✅ **Правило.** Тримайте `DEPLOY.md`, `CHANGELOG.md` і живий SEO-план у репозиторії поруч із кодом. У нас `CHANGELOG.md` розрісся до 96 КБ і саме він рятує при поверненні до задачі через два місяці — без нього причини половини рішень втрачаються.

2. База даних: повна схема

Файл: `backend/db/index.js` . Виконується **при кожному старті процесу** — створює таблиці, доливає колонки, засіює порожні таблиці, самовиправляє дані.

```
const db = new Database(DB_PATH)
db.pragma('journal_mode = WAL') // паралельні читання під час запису
db.pragma('foreign_keys = ON')
```

2.1 `products` — центральна таблиця

Це єдина таблиця, від якої залежить усе інше: каталог, SEO, фіди, прайс, кошик.

Колонка	Тип	Призначення й підводні камені
<code>id</code>	TEXT PK	рядковий, не автоінкремент. Формат <code>p_<timestamp></code> . Використовується в кошику й GA4 як <code>item_id</code> -фолбек
<code>wp_id</code>	INTEGER	ID зі старого WordPress. Тримали для зв'язки при міграції; у новому проєкті не потрібен
<code>name</code>	TEXT NOT NULL	канонічна українська назва. Показується в каталозі, картці, листі, Telegram
<code>slug</code>	TEXT UNIQUE NOT NULL	частина URL <code>/catalog/:cat/:slug</code> . Змінювати після індексації не можна без 301
<code>sku</code>	TEXT	артикул. Джерело істини для зв'язки з 1С і для <code>g:id</code> у фідах
<code>price</code>	REAL	ціна в рідній валюті позиції. <code>0</code> або NULL = «ціна по запиту»
<code>currency</code>	TEXT DEFAULT 'UAH'	<code>UAH</code> або <code>EUR</code> . Керує конвертацією (розділ 10.1)
<code>sale_price</code>	REAL DEFAULT 0	акційна ціна. <code>0</code> = без акції. Якщо <code>< price</code> — товар автоматично попадає в категорію «Акція»
<code>category_slug</code>	TEXT	slug категорії з довідника (розділ 5). Не FK — довідник живе в кодi
<code>subcategory</code>	TEXT	текстова підкатегорія для фільтрів
<code>image</code>	TEXT	головне фото. Або <code>/uploads/...</code> , або абсолютний URL
<code>images</code>	TEXT (JSON)	масив додаткових фото. Віддається в <code>Product.image[]</code> схеми
<code>short_desc</code>	TEXT	короткий опис під назвою в картці товару. Фолбек для <code>description</code> у фідах
<code>description</code>	TEXT	повний опис у HTML (<code><p></code> , <code></code> , <code><a></code> , <code></code>) — 300–450 слів (розділ 7)
<code>seo_title</code>	TEXT	★ готовий <code><title></code> , ≤ 60 символів, унікальний, з брендом. Не генерується — пишеться
<code>meta_description</code>	TEXT	★ готовий <code><meta name=description></code> , 140–160 символів

Колонка	Тип	Призначення й підводні камені
<code>specs</code>	TEXT (JSON)	<code>{"Потужність":"15 кВт","Підключення":"1\\" BP"}</code> — джерело таблиці характеристик, фільтрів каталогу і SSR-таблиці
<code>features</code>	TEXT (JSON)	масив коротких тез-переваг
<code>in_stock</code>	INTEGER DEFAULT 1	наявність → <code>availability</code> у схемі й фідах
<code>is_visible</code>	INTEGER DEFAULT 1	★ показ на сайті. <code>0</code> = чернетка: не в каталозі, не в sitemap, не у фідах, віддає 404
<code>i18n</code>	TEXT (JSON)	переклади: <code>{"en":{"...},"pl":{"...},"fr":{"...},"de":{"...}}</code> (розділ 9)
<code>created_at</code>	TEXT	<code>datetime('now')</code> . Використовується як <code>lastmod</code> у sitemap

✅ **Правило (найважливіше у розділі).** `seo_title` і `meta_description` — окремі поля БД, а не похідні від назви. Саме це дало нам «269 задовгих title → 0» одним прогоном. Автогенерація з назви завжди дає або дублі, або обрізання в SERP.

⚠️ **Граблі:** `is_visible` ≠ `in_stock` . Це різні речі, і плутанина коштує трафіку. `in_stock=0` — товар є в каталозі, індексується, має ціну, але позначений «під замовлення». `is_visible=0` — товару **немає для світу**: він зникає з каталогу, з sitemap, з фідів, а його URL віддає 404. Знімати з продажу індексований товар через `is_visible=0` можна лише свідомо (URL випадає з індексу); частіше правильніше лишити видимим з `in_stock=0` .

2.2 Решта таблиць

Таблиця	Ключові поля	Призначення
<code>orders</code>	<code>items</code> (JSON), <code>total</code> , <code>name</code> , <code>phone</code> , <code>email</code> , <code>address</code> , <code>comment</code> , <code>payment</code> , <code>status</code> , <code>utm</code> (JSON), <code>np_city</code> , <code>np_city_ref</code> , <code>np_warehouse</code> , <code>np_warehouse_ref</code>	замовлення з кошика. <code>total</code> завжди перераховується сервером
<code>consultations</code>	<code>name</code> , <code>phone</code> , <code>email</code> , <code>message</code> , <code>status</code> , <code>utm</code>	форма «Консультація»
<code>dealers</code>	<code>company</code> , <code>name</code> , <code>phone</code> , <code>email</code> , <code>city</code> , <code>message</code> , <code>status</code> , <code>utm</code>	заявка «Стати дилером»
<code>blog_posts</code>	<code>slug</code> UNIQUE, <code>title</code> , <code>excerpt</code> , <code>content</code> , <code>image</code> , <code>tags</code> , <code>category</code> , <code>published</code> , <code>published_at</code> , <code>i18n</code>	блог. <code>published=0</code> → не в sitemap, URL → 404
<code>portfolio</code>	<code>title</code> , <code>location</code> , <code>year</code> , <code>power</code> , <code>type</code> , <code>description</code> , <code>images</code> , <code>links</code> , <code>i18n</code>	реалізовані об'єкти — сильний E-E-A-T-сигнал
<code>reviews</code>	<code>name</code> , <code>company</code> , <code>rating</code> , <code>text</code> , <code>photo</code> , <code>published</code> , <code>i18n</code>	відгуки про компанію
<code>product_reviews</code>	<code>product_id</code> , <code>product_name</code> , <code>product_slug</code> , <code>category_slug</code> , <code>name</code> , <code>rating</code> , <code>text</code> , <code>photo</code> , <code>published</code>	відгуки про товар , <code>published=0</code> = на модерації
<code>faqs</code>	<code>question</code> , <code>answer</code> , <code>sort</code> , <code>i18n</code>	21 Q&A → сторінка FAQ і <code>FAQPage</code> JSON-LD
<code>files</code>	<code>name</code> , <code>filename</code> , <code>category</code> , <code>size</code>	документація (PDF-каталоги, інструкції)

Таблиця	Ключові поля	Призначення
subscribers	email UNIQUE	підписка + автозбір e-mail із замовлень
banners	title, subtitle, image, link, active, sort	банери
promos	title, description, discount, image, active, sort	акції
clients	name, logo, url, sort	логотипи клієнтів
settings	key PK, value	контакти, робочі години, пароль адміна, контентні блоки

⚠ **Габлі:** два види відгуків — і це навмисно. `reviews` (про компанію) Google **не показує зірками** в результатах — політика проти self-serving review snippets. Зірки біля товару дає лише `AggregateRating`, побудований з відгуків **на конкретний товар**. Тому таблиці розділені: 5 відгуків про компанію підняли нам E-E-A-T 61 → 68, але зірок не дали. Для зірок потрібні per-product відгуки.

⚠ **Габлі:** `settings` **віддається публічно**. `GET /api/settings` не вимагає авторизації (сайту потрібні телефон і адреса). Роут вирізає лише `adminPassword`. ✅ **Правило: жодного секрету в `settings`**. Токен Telegram-бота, ключі API, паролі — **тільки в ENV**. Ми окремо задокументували це в коментарі `backend/telegram.js`, бо спроба «покласти токен у налаштування, щоб міняти з адмінки» — природна й фатальна.

2.3 Патерн міграцій

Немає ORM і немає файлів міграцій. Замість них — **ідемпотентні операції при кожному старті**:

```
// Додати колонку: якщо вона є — ALTER падає, помилку ковтаємо
try { db.exec(`ALTER TABLE orders ADD COLUMN utm TEXT DEFAULT '{}'`) } catch (e) { /* існує */ }

// Або через перевірку схеми (чистіше)
const cols = db.prepare('PRAGMA table_info(products)').all().map(c => c.name)
if (!cols.includes('seo_title')) db.exec("ALTER TABLE products ADD COLUMN seo_title TEXT DEFAULT ''")
```

Окремий клас — **самовиправні міграції**, які приводять дані до інваріанта щостарту. Приклад: `specs.Артикул` мусить дорівнювати `sku` (бо артикул редагується в двох місцях і розходився):

```
if (specs['Артикул'] !== undefined && specs['Артикул'] !== r.sku) {
  specs['Артикул'] = r.sku // sku — джерело істини
  upd.run(JSON.stringify(specs), r.id)
}
```

✅ **Правило.** Кожна міграція мусить бути безпечною при 100-му запуску. Це дає властивість, яка на практиці цінніша за красу: **перезапуск контейнера самолікує дані**.

⚠ **Габлі:** одна міграція **безумовно перезаписує дані з seed**. `migrateCurrency()` щостарту пропоставляє `currency` з `seed-products.json` для всіх товарів `seed'y`, трактуючи файл як джерело істини. Тобто зміна валюти прямо в живій БД **відкотиться при наступному рестарті**, якщо не змінити

ще й seed. Ми свідомо це лишили (валюта рідко змінюється), але механізм треба знати. У новому проєкті краще: seed сіє лише те, чого немає, і **ніколи** не перезаписує наявні рядки.

2.4 Сідинг

```
function seedProducts() {
  const count = db.prepare('SELECT COUNT(*) as c FROM products').get().c
  if (count > 0) return // ← сіємо ЛИШЕ в порожню таблицю
  ...
}
```

Так само `seedBlog()` і `seedPortfolio()`. Це дає self-healing: якщо таблиця `portfolio` колись опустіла, наступний старт відновить її з `seed-portfolio.json`, і сторінка «Наші проєкти» не буде порожньою. Але правки з адмінки при цьому не перезаписуються — бо коли записи є, сідинг не робить нічого.

⚠ **Граблі: те, що ви правите в живій БД, у git не потрапляє.** Файл БД лежить у volume. Дедуплікацію `seo_title` ми зробили прямо в продовій БД — і повний ре-деплой «з нуля з seed» її втратив. Тому всі масові правки даних у нас оформлені **ідемпотентними скриптами в git** (`backend/scripts/apply-*.js`, `scripts/seo-fix-titles.cjs`), які можна прогнати повторно (розділ 15).

3. API: повний перелік і правила

Файл: `backend/server.js` — реєстрація роутерів; `backend/routes/*` — реалізація.

3.1 Карта ендпоінтів

Шлях	Метод	Доступ	Що робить
<code>/api/auth/login</code>	POST	публічний, ліміт 10/15хв	пароль → JWT на 24 год
<code>/api/products</code>	GET	публічний	список; <code>?category=</code> , <code>?search=</code> , <code>?page=</code> , <code>?limit=</code> (деф. 100)
<code>/api/products?admin=1</code>	GET	адмін	+ приховані (<code>is_visible=0</code>)
<code>/api/products/:slug</code>	GET	публічний	один товар за slug
<code>/api/products</code>	POST/PUT/DELETE	адмін	CRUD товару
<code>/api/orders</code>	POST	публічний, 20/хв	замовлення: БД + Telegram + CRM + лист
<code>/api/orders</code>	GET / PUT	адмін	список / зміна статусу
<code>/api/consultations</code> , <code>/api/dealers</code>	POST	публічний, 20/хв	форми-ліди → БД + Telegram + CRM
<code>/api/blog</code> , <code>/api/portfolio</code> , <code>/api/faq</code> , <code>/api/files</code> , <code>/api/banners</code> , <code>/api/promos</code> , <code>/api/clients</code>	GET / CRUD	GET публічний, запис — адмін	контентні сутності
<code>/api/reviews</code>	POST / CRUD	публічний POST	відгуки про компанію
<code>/api/product-reviews</code>	GET	публічний	лише <code>published=1</code>
<code>/api/product-reviews/submit</code>	POST	публічний, 20/хв	відгук на товар + фото → на модерацію
<code>/api/product-reviews/:id</code>	PUT/DELETE	адмін	модерація
<code>/api/subscribers</code>	POST / GET	POST публічний	підписка
<code>/api/settings</code>	GET	публічний	контакти (без <code>adminPassword</code>)
<code>/api/settings</code>	PUT	адмін	білий список ключів + хешування пароля
<code>/api/analytics</code>	GET	адмін	зведення: ліди по днях, UTM, статуси
<code>/api/np/cities</code> , <code>/api/np/warehouses</code>	GET	публічний, 60/хв	проксі Нової Пошти

Шлях	Метод	Доступ	Що робить
<code>/api/upload</code>	POST	адмін	файл → <code>/uploads/...</code>
<code>/api/webhooks/binotel</code>	POST	IP-allowlist, 120/хв	вхідний дзвінок → CRM + Telegram
<code>/google-merchant{-en,-pl,-de,-fr}.xml</code>	GET	публічний	фіди Google Shopping, кеш 1 год

3.2 Авторизація

Одна роль — адміністратор. Один пароль у `settings.adminPassword`.

```
const JWT_SECRET = process.env.JWT_SECRET
if (!JWT_SECRET || JWT_SECRET.length < 32) {
  throw new Error('JWT_SECRET not set or too short (must be ≥ 32 chars)')
}
```

✅ **Правило: fail fast.** Контейнер **не стартує** без нормального секрету. Це краще, ніж тихо працювати з дефолтним ключем — саме така помилка є найпоширенішою причиною зламаних адмінок.

Хешування з м'якою міграцією. Історично пароль лежав відкритим текстом. Замість ризикованого разового скрипту зробили міграцію «при першому вході»:

```
if (storedValue.startsWith('$2')) { // уже bcrypt
  valid = bcrypt.compareSync(password || '', storedValue)
} else { // legacy plaintext
  valid = password === storedValue
  if (valid) db.prepare("INSERT OR REPLACE INTO settings ...")
    .run(bcrypt.hashSync(password, 12)) // ← мігруємо на місці
}
```

Дві перевірки токена — навмисно: - `requireAdmin` — жорстка, повертає 401 (усі записи); - `checkAdmin(req)` — м'яка, повертає `true/false` і **не блокує** запит. Потрібна там, де адмін бачить більше на тому ж ендпоінті: `/api/products?admin=1` віддає приховані товари, `/api/product-reviews?admin=1` — немодеровані.

⚠️ **Габлі.** Токен зберігається в `sessionStorage` (`tj2_admin_token`), а не в `localStorage`: закриття вкладки завершує сесію. Термін життя JWT скорочено з 7 днів до 24 годин.

💡 **Зробити краще.** Для магазину з кількома співробітниками одного пароля мало. Мінімум для нового проекту: таблиця користувачів, роль (`admin` / `manager` / `content`), журнал дій (хто змінив ціну і коли).

3.3 Rate limiting

```
loginLimiter   - 10 / 15 хв // антибрутфорс
writeLimiter   - 20 / хв   // публічні POST: замовлення, форми, відгуки
npLimiter      - 60 / хв   // проксі Нової Пошти
binotelLimiter - 120 / хв  // вебхук дзвінків: на дзвінок кілька подій
```

⚠ **Габлі, які варті окремого абзацу:** `trust proxy` за `nginx` у `Docker`. Усередині контейнера `peer` — це `docker-gateway` (172.x), а не `loopback`. Без налаштування `Express` брав IP шлюза для **всіх** запитів: `rate-limit` ставав глобальним, а `IP-allowlist` вебхука `Binotel` відхиляв **усі** дзвінки.

```
app.set('trust proxy', ['loopback', 'uniquelocal'])
```

Ланцюг довірених приватних проксі пройдено → `req.ip` = справжній клієнтський IP з `X-Forwarded-For`. Зовнішній клієнт (публічний `peer`) не довірений, тому підробити `XFF` не може.

✅ **Правило.** Ставте `trust proxy` до будь-якого `rate-limit` чи `IP-фільтра` і перевіряйте живим запитом, а не припущенням.

3.4 Ніколи не довіряти клієнту в ціні

`POST /api/orders` ігнорує ціни й суму з тіла запиту й перераховує все з БД:

```
const eurRate = await getEurRate()
for (const item of items) {
  const row = getProduct.get(item.id) // ціна з БД, не з клієнта
  if (!row) continue // неіснуючий id просто пропускаємо
  const qty = Number(item.qty ?? item.quantity) || 1
  serverTotal += toUAH(row.price, row.currency, eurRate) * qty
}
```

⚠ **Габлі, які реально стріляли.** Кошик фронтенду кладе кількість у поле `quantity`, а бекенд читав `qty` → у `Telegram` і листах усі позиції були «x 1», а сума не збігалася. Фікс — читати обидва: `item.qty ?? item.quantity`. ✅ **Правило:** узгодьте назви полів кошика й `API` одним словником з першого дня.

Другий бік тієї ж помилки: `EUR`-товари. Без конвертації сума «в грн» була числом у євро — приблизно в 52 рази менша за реальну. Тому конвертація живе і на фронті, і на бекенді (розділ 10.1).

3.5 Обробка помилок

```
app.use((err, req, res, next) => {
  console.error(err) // повний стек — у логи сервера
  res.status(err.status || 500).json({ error: 'Internal error' }) // клієнту — нічого
})
```

✅ **Правило.** `Стектрейси` не виходять за периметр: клієнт отримує загальне повідомлення, деталі — тільки в `docker logs`.

4. Адмінка: розділи, поля, реальні межі

Вхід: `/admin` (окремий layout без сайтового хедера й футера). Роути адмінки виключені з індексації через `robots.txt` (`Disallow: /admin`) і не віддають SEO-метадані.

4.1 Двадцять розділів

Розділ	Роут	Що редагує	Таблиця
Дашборд	<code>/admin/dashboard</code>	зведення й швидкі переходи	—
Товари	<code>/admin/products</code>	каталог: створення, правка, приховування, видалення	<code>products</code>
Замовлення	<code>/admin/orders</code>	склад, контакти, доставка, статус	<code>orders</code>
Консультації	<code>/admin/consultations</code>	заявки з форми зв'язку	<code>consultations</code>
Дилери	<code>/admin/dealers</code>	заявки на партнерство	<code>dealers</code>
Блог	<code>/admin/blog</code>	статті: заголовок, анонс, контент, категорія, дата, публікація	<code>blog_posts</code>
Портфоліо	<code>/admin/portfolio</code>	об'єкти: назва, локація, рік, потужність, тип, фото, лінки	<code>portfolio</code>
Файли	<code>/admin/files</code>	документація для сторінки «Файли»	<code>files</code>
Відгуки	<code>/admin/reviews</code>	відгуки про компанію	<code>reviews</code>
Відгуки на товари	<code>/admin/product-reviews</code>	модерація відгуків з сайту	<code>product_reviews</code>
FAQ	<code>/admin/faq</code>	питання/відповіді + порядок	<code>faqs</code>
Банери	<code>/admin/banners</code>	банери	<code>banners</code>
Акції	<code>/admin/promos</code>	акційні блоки	<code>promos</code>
Підписники	<code>/admin/subscribers</code>	е-mail-база	<code>subscribers</code>
Аналітика	<code>/admin/analytics</code>	ліді по днях, дохід, статуси	агрегація
UTM	<code>/admin/utm</code>	джерела/канали/кампанії лідів	агрегація
Контент головної	<code>/admin/content</code>	тексти блоків головної	<code>settings.homeContent</code>
Про компанію	<code>/admin/about</code>	тексти й фото сторінки «Про нас»	<code>settings.aboutContent</code>
Налаштування	<code>/admin/settings</code>	телефон, е-mail, адреса, години, пароль	<code>settings</code>
Логін	<code>/admin</code>	вхід	—

4.2 Форма товару: що є в UI і чого в ньому немає

Поля, доступні в адмінці:

Поле	Тип у формі	Примітка
Назва *	текст	обов'язкове; з нього генерується slug, якщо не заданий
Артикул (SKU)	текст	звірка з 1C, <code>g:id</code> у фідах
Ціна (грн)	число	<code>0</code> = «ціна по запиту» (підказка є в UI)
Акційна ціна	число	порожньо = без акції; менша за ціну → товар у категорії «Акція»
Категорія *	select із довідника	15 варіантів із <code>CATEGORIES</code>
Фото товару	завантаження	компонент <code>ImageUpload</code> → <code>/api/upload</code>
Опис	textarea	приймає HTML
В наявності	чекбокс	→ <code>in_stock</code>
Показувати на сайті	чекбокс	→ <code>is_visible</code>
Характеристики	пари «Параметр / Значення»	динамічний список → JSON <code>specs</code>

Slug генерується автоматично з назви (`toSlug()`), якщо не заданий явно:

```
const slug = form.slug || toSlug(form.name)
```

⚠ **Габлі — і це найважливіша правда про нашу адмінку.** Частина полів БД **не має UI** і редагується лише скриптами або прямо в БД:

Поле	Чому це болить
<code>seo_title</code>	головне SEO-поле товару — контент-менеджер не може його змінити
<code>meta_description</code>	те саме
<code>images</code> (галерея)	у формі лише головне фото
<code>currency</code>	EUR-товари заводяться тільки скриптом
<code>features</code>	масив переваг
<code>subcategory</code>	впливає на фільтри каталогу
<code>i18n</code> (4 мови)	усі переклади — лише через скрипти

💡 **Зробити краще — пункт №1 для нового магазину.** Додайте у форму товару вкладки: 1. **Основне** — те, що є зараз; 2. **SEO** — `seo_title` (з лічильником символів і жорстким лімітом 60), `meta_description` (лічильник, цільовий діапазон 140–160), прев'ю сніпета Google; 3. **Галерея** — мультизавантаження з перетягуванням; 4. **Переклади** — таби мов із тими самими полями; 5. **Ціна** — валюта позиції, акційна ціна, дати акції.

Без цього кожна SEO-правка перетворюється на технічну задачу для розробника — у нас саме так і вийшло: 331 опис заводився конвеєром скриптів (розділ 7).

4.3 Характеристики (`specs`) — не просто текст

`specs` виглядає як довільний JSON, але фактично це **структуровані дані, на яких тримається три механізми:**

1. таблиця «Технічні характеристики» на сторінці товару;
2. **фільтри каталогу** — значення витягуються саме зі `specs` (розділ 5.4);
3. таблиця характеристик у SSR-блоці для краулерів (розділ 6.4).

✅ **Правило: однакові ключі для однакових параметрів у межах категорії.** Якщо в одному товарі «Потужність, кВт», а в іншому «Максимальна потужність» — фільтр розпадається на дві незалежні групи, і користувач не знайде товар.

⚠️ **Габлі з юнікодом.** У нас у ключах трапляються різні варіанти `Δ / Δ` і латинська `T` замість кириличної `Т` (`ΔT=10` vs `ΔT=10`). Тому матчери фільтрів шукають **підрядки** («10», «20») замість точного порівняння ключів. Це працює, але це лікування симптому. ✅ **Правило: заведіть словник дозволених ключів характеристик по категорії** й валідуйте при збереженні товару.

Ще один інваріант: `specs["Артикул"]` синхронізується з `sku` **щостарту** (розділ 2.3) — писати артикул у характеристики руками не потрібно, значення перезапишеться.

4.4 Модерація відгуків на товар

Публічний потік:

```
POST /api/product-reviews/submit (multipart: текст + фото)
├ валидація: name ≤ 80; text 10...1500 символів; rating 1-5 (інакше 5)
├ фото: jpg|jpeg|png|webp, ліміт 8 МБ → /uploads/preview-<ts>--<rand>.<ext>
├ INSERT ... published = 0          ← ⚠️ НЕ показується, доки не схвалить
└ відповідь: «Дякуємо! Відгук з'явиться після перевірки модератором.»
```

У рядок кладеться **знімок товару** (`product_name`, `product_slug`, `category_slug`) — щоб адмінка показувала «до якого товару» без JOIN, і щоб перейменування товару не ламало історію.

✅ **Правило.** Публічні відгуки — тільки з премодерацією. Форма відгуку — це відкритий ендпоінт запису; без `published=0` і `rate-limit` це канал для спаму.

4.5 Завантаження файлів

```
limits: { fileSize: 25 * 1024 * 1024 } // було 100 МБ — знизили
fileFilter: /\. (jpg|jpeg|png|webp|gif|pdf|doc|docx|xls|xlsx|ppt|pptx|dwg|zip|mp4|webm|mov|m4v)$/i
```

⚠️ **Габлі: SVG прибрано з whitelist навмисно.** SVG — це XML, який може містити `<script>`. Файл, що віддається з нашого домену, дає **stored XSS** із доступом до адмінської сесії. Іконки та ілюстрації тримаємо у білді, а не в завантаженнях.

Ім'я файлу завжди генерується (`<timestamp>--<random>.<ext>`) — оригінальна назва не використовується (обхід шляху, кириличні імена, колізії).

4.6 Чого в адмінці немає — свідомо чи через борг

Немає	Наслідок	Пріоритет для нового магазину
SEO-полів у формі товару	правки лише скриптами	🔴 критично
редактора перекладів	4 мови обслуговуються вручну	🔴 критично, якщо мов більше однієї

Немає	Наслідок	Пріоритет для нового магазину
ролей і журналу дій	не видно, хто змінив ціну	🟡 важливо
масового редагування / CSV-імпорту	зміна цін по категорії = ручна робота	🔴 критично при 300+ SKU
керування категоріями	довідник у кодї (розділ 5.2)	🟡 важливо
керування редиректами	<code>redirects.json</code> у git	🟢 бажано
перегляду складських залишків	наявність — ручний чекбокс	🟡 залежить від 1С

5. Каталог: структура і таксономія

5.1 П'ятнадцять категорій

Довідник: `src/data/categories.js`. Кожна категорія має `id`, `slug`, іконку, фото, колір, **назву й опис 5 мовами** та **список підкатегорій 5 мовами**.

#	slug	Назва (UA)	Підкатегорії (UA)
1	nasosni-hrupy	Насосні групи	з термостатичним клапаном · з теплоізоляцією EPP · без теплоізоляції
2	hidravlichni-rozdilnyky	Роздільники гідравлічні	ГС до 60 / 105 / 175 кВт · ГС Mega
3	rozpodilchi-kolektory	Розподільчі колектори	серії 60 / 105 / 175 кВт · Mega
4	kolektory-z-hidrostrilkoyu	Колектори з гідрострілкою	2- / 3- / 4- / 5-6-контурні
5	termojet-box	Модульні системи TERMOJET BOX	BOX 2 контури · BOX 3 контури · BOX з ГВС-бойлером
6	termojet-mega	Серія Termojet Mega (до 2200 кВт)	до 600 кВт · до 1 МВт · до 2.2 МВт
7	nasosy	Насоси	APM · XPS · Grundfos · Wilo
8	klapany	3-х/4-х ходові та термостатичні клапани	3-ходові · 4-ходові · електричні сервоприводи
9	balansuval-klapany	Статичний балансувальний клапан	статичні · з вимірювальним ніпелем
10	separatory	Сепаратори	шламові · повітряні · комбіновані
11	zonalne-keruvannya	Термостати та зональне керування	програматори дротові / бездротові · центри комутації · термоголовки й датчики
12	kolektory-pidloha	Система підлогового опалення	гребінки · з витратомірами · змішувальні вузли · шафи
13	avtomatyka	Автоматика котельного обладнання	Termojet Light · Profi Plus · погодозалежне управління
14	dodatkovye	Додаткове обладнання	кріплення · теплоізоляція · шафи · аксесуари
15	rozprodazh	Акція	— (збірна)

✅ **Правило: пласка структура, один рівень.** Категорія → товар, без вкладених підкатегорій-URL. Підкатегорія — це **атрибут товару** (`subcategory`), який працює як фільтр, а не як окрема сторінка. Це дає короткі URL, відсутність дублів і просту перелінковку. Для асортименту опалювального обладнання (кілька сотень SKU у 15 групах) одного рівня достатньо.

Категорія «Акція» — віртуальна. Вона збирає товари, у яких заповнена `sale_price`, менша за `price`, плюс явно призначені. Окремо переносити товар не треба — достатньо вказати акційну ціну.

5.2 ⚠ Головні граблі каталогу: довідник категорій існує у трьох копіях

Бекенд написаний на CommonJS і **не імпортує ESM-модулі фронтенду**. Через це назви й описи категорій дублюються:

Копія	Файл	Що з неї бере користувач/бот
1	<code>src/data/categories.js</code> → <code>CATEGORIES</code>	видимі назви, описи, підкатегорії, фото (5 мов)
2	<code>backend/server.js</code> → <code>CATEGORY_META</code>	<code><title></code> , <code>description</code> , H1, breadcrumbs, SSR-блок (UA + EN)
3	<code>backend/routes/merchant.js</code> → <code>CAT</code>	<code>product_type</code> у фідах Google Shopping (5 мов)

Змінили назву або опис категорії — оновіть усі три. Ми зафіксували це окремим попередженням у `DEPLOY.md`, бо розсинхронізація дає найгірший з можливих результатів: користувач бачить один текст, Google — інший.

💡 **Зробити краще.** Тримайте категорії в **БД** (таблиця `categories` з локалізованими полями) або в спільному JSON, який читають і фронт, і бекенд. Три ручні копії — головний технічний борг нашого каталогу.

5.3 URL-схема

```
/catalog                — уся сітка категорій
/catalog/:categorySlug  — сторінка категорії
/catalog/:categorySlug/:productSlug — сторінка товару
/en/catalog/... /pl/... /fr/... /de/... — мовні дзеркала того самого дерева
```

✅ **Правила слів:** - **транслітерація українською**, не переклад: `kolektor-k21v-125`; - slug містить модель — це і читабельність, і ключ; - **жодних числових слів**. У нас 19–24 таких лишилось зі старого сайту (`84312520a`) — виправити їх тепер означає 301 + ризик колізій, тому свідомо відкладено. У новому магазині це просто не допускається; - довжина URL до 100 символів; - slug не змінюється після індексації. Треба змінити — робіть 301.

⚠ **Граблі з мовними слагами.** Спочатку `/en/` віддавав сторінки з **польськими** слагами зі старого WordPress (`/en/product/zawor...`) — індексувались дублі. Зараз правило одне: **slug однаковий для всіх мов**, змінюється лише мовний префікс. Перекладати slug — множити проблеми.

5.4 Фільтри каталогу — витягуються з характеристик

Фільтри в сайдбарі **не мають окремої моделі даних**: `CatalogPage.jsx` розбирає `specs` товарів поточної категорії й будує групи фільтрів під цю категорію. Плюс — ціновий діапазон, який рахується по фактичних цінах категорії.

Приклади реальних витягачів:

```
// Продуктивність насоса: значення в л/хв нормалізуємо до м³/год
// Напір насоса (м) — зі specs
// Тип приєднання: з назви + полів specs про з'єднання/діаметр/вихід
// Кількість балок колектора: друга цифра коду K[розмір][балки]...
// 1 = однобалковий (Mini), 2 = двобалковий. Перевірено на всіх 41 товарі: 7/34 без перетинів
```

Тобто частина класифікації **виводиться з артикула/назви за угодою про нумерацію**. Це працює, поки нумерація дисциплінована.

Також задані **кастомні порядки сортування** всередині категорій, бо алфавіт для техніки безглуздий: - тепла підлога: змішувальні вузли → колектори з витратомірами (2→15) → з кранами (2→12) → шафи → інше; - сепаратори: повітря → повітря+бруд → бруд → інше; всередині — за серією і DN.

✅ **Правило.** Порядок товарів у категорії — це UX-рішення, яке має бути описане в коді явно. За замовчуванням «за назвою» ставить K10 після K1 і плутає покупця.

💡 **Зробити краще.** Виведіть фільтри в окрему структуру: `{ категорія: [{ ключ_specs, тип (діапазон/множинний вибір), одиниця, порядок }] }`. Тоді додавання фільтра — це рядок конфігурації, а не правка компонента на 1600 рядків.

5.5 Ціна «по запиту»

`price = 0` або NULL — легальний стан: товар показується, але замість ціни — «ціна по запиту».

⚠️ **Габлі, які ловив аудит.** Товар без ціни віддавав `Product` -схему **без** `offers` — такі схеми невалідні (Google вимагає `offers / review / aggregateRating`), і ми отримали 5 помилок структурованих даних. Фікс: для товарів без ціни віддаємо лише `BreadcrumbList` + `Organization`, без `Product` (розділ 6.5).

6. SEO: повна механіка

Найбільш пророблена частина проєкту. Тут описано **все**, що віддається краулеру, і чому саме так.

6.1 Оболонка `index.html` як шаблон для заміни

`index.html` містить **дефолтні значення**, які працюють як **маркери підстановки**. Сервер робить рядкову заміну по них.

```
<title>Termojet – Виробник обладнання для котелень</title>
<meta name="description" content="Termojet – український виробник..." />
<link rel="canonical" href="https://termojet.com.ua/" />
<meta name="robots" content="index, follow" />
<meta property="og:type" content="website" />
<meta property="og:site_name" content="Termojet" />
<meta property="og:locale" content="uk_UA" />
<meta property="og:url" content="https://termojet.com.ua/" />
<meta property="og:title" content="..." />
<meta property="og:description" content="..." />
<meta property="og:image" content="https://termojet.com.ua/images/portfolio/proj-1.jpg" />
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="..." />
<meta name="twitter:description" content="..." />
```

У тілі — два контейнери:

```
<div id="root"></div>          <!-- React -->
<div id="seo-content"></div>    <!-- сюди сервер вкладає семантичний HTML -->
```

⚠ **Габлі: заміна прив'язана до точних дефолтних рядків.**

```
out = html.split(DEFAULT_TITLE).join(esc(title))      // за текстом заголовка
out = img ? html.split(DEFAULT_OG_IMG).join(esc(img)) : html  // за URL картинки
```

Тобто якщо змінити дефолтний `<title>` або дефолтний `og:image` в `index.html` і не змінити константи `DEFAULT_TITLE` / `DEFAULT_OG_IMG` у `server.js` — **ін'єкт тихо перестане працювати**. Помилки не буде: сторінки просто почнуть віддавати generic-метадані.

✅ **Правило для нового магазину.** Використовуйте явні маркери, які неможливо випадково зламати: `<title>{{TITLE}}</title>`, `<!--SEO_HEAD-->`, `<!--SEO_BODY-->`. І додайте smoke-тест: `curl` головної й товару з перевіркою, що в HTML немає `{{` і немає дефолтного заголовка.

6.2 Що ін'єктиться в `<head>`

Функція `injectMeta(html, { title, desc, url, img, ogType, h1, bodyText, alternates, jsonLd })`:

Тег	Джерело значення
<code><title></code>	<code>seo_title</code> з БД; фолбек <code>{name} \ Termojet</code>
<code><link rel="canonical"></code>	self-canonical — точний URL цієї сторінки
<code><link rel="alternate" hreflang></code>	<code>uk</code> , <code>en</code> , <code>x-default</code> — вставляються одразу після canonical
<code><meta name="description"></code>	<code>meta_description</code> ; фолбек: <code>short_desc</code> → <code>description</code> (обрізка 200)
<code>og:url</code> , <code>og:type</code> , <code>og:description</code> , <code>og:image</code>	те саме, <code>og:type=article</code> для блогу
<code>twitter:description</code>	те саме
<code><script type="application/ld+json"></code>	схеми (розділ 6.5), вставляються перед <code></head></code>

⚠ **Граблі: «стрей `/>`» угорі сайту.** Перша версія регулярки замінювала `canonical` без кінцевого `/>`, і залишок `/>` вивалювався у видимий текст сторінки, а перший `hreflang` «проковтувався» незакритим тегом. Правильна заміна — **тега цілком**:

```
.replace(/<link rel="canonical" href="[^"]*"s*\/?>/, () => { ... })
```

✅ **Правило.** Заміна тегів регулярками — робоче рішення, але кожна заміна мусить покривати тег **цілком** і бути перевірена на живому HTML (`curl | grep`), а не лише в юніт-тесті.

6.3 Локалізація метаданих: `pickLang`

```
function pickLang(row, lang) {
  if (!lang || lang === 'uk') return row // UA — канонічні колонки
  const i18n = JSON.parse(row.i18n || 'null')
  if (!i18n?.[lang]) return row // немає перекладу → фолбек на UA
  return Object.assign({}, row, {
    name: loc.name || row.name,
    description: loc.description || row.description,
    seo_title: loc.seo_title || row.seo_title,
    meta_description: loc.meta_description || row.meta_description,
    ...
  })
}
```

Фолбек **пофайлово на кожне поле**: якщо перекладено назву, але не мета — мета візьметься українська. Це гірше за повний переклад, але значно краще за порожній тег.

6.4 SSR-lite: блок `#seo-content`

Метадані — це `<head>`. Але аудит фіксував ще «thin content» і «orphan pages»: у сирому HTML **не було тіла сторінки**. Рішення — сервер генерує семантичний HTML сторінки і вкладає його в `#seo-content`.

Приховування — чистий CSS, без мигання для користувача:

```
#root:not(:empty) ~ #seo-content { display: none; }
```

Щойно React заповнив `#root`, блок зникає (додатково `main.jsx` його прибирає). Краулер без JS бачить повний контент.

Що всередині для сторінки товару:

```
<div id="seo-content">
  <nav>Головна / Каталог / {Категорія}</nav>           ← breadcrumbs лінками
  <h1>{назва}</h1>
  <img src alt width="360" height="360" loading="lazy">
  <p>Артикул: {sku}</p>
  <p><strong>Ціна: {N} грн</strong></p>                  ← EUR уже конвертовано
  <p>Наявність: В наявності | Під замовлення</p>
  <div>{повний HTML-опис}</div>
  <h2>Технічні характеристики</h2>
  <table>...                                           ← зі specs, крім «Артикул»/«Назва»
  <h2>Схожі товари</h2><ul>...                       ← 4 товари категорії з цінами
  <nav aria-label="Основні розділи">...               ← 9 лінків на розділи
</div>
```

Для інших типів сторінок:

Сторінка	Вміст <code>#seo-content</code>
Категорія	breadcrumbs, H1, опис, <code><h2>Товари категорії (N)</h2></code> + усі товари лінками з цінами, навігація
Головна	інтро-абзац + <code><h2>Каталог обладнання</h2></code> + усі 15 категорій із описами
Блог (список)	<code><h2>Статті (N)</h2></code> + усі опубліковані статті з анонсами (180 символів)
Стаття	H1 + повний HTML-контент статті + навігація
FAQ	усі питання <code><h3></code> + відповіді <code><p></code>
Про нас / Сервіс / Доставка / OEM / Партнери / Повернення / Навчання / Портфоліо	стислий фактичний текст із мапи <code>PAGE_CONTENT</code>

✅ **Правило: цей блок лікує «orphan pages».** Кожна сторінка товару отримує 4 лінки на схожі товари + 9 на розділи; сторінка категорії — лінки на всі свої товари. Так внутрішня перелінковка з'являється в сирому HTML, а не лише в відрендереному DOM.

⚠️ **Габлі:** `express.static` мусить мати `index: false`.

```
app.use(express.static(DIST, { redirect: false, index: false, setHeaders: ... })))
```

- `index: false` — інакше статика сама віддасть `index.html` для `/`, і catch-all ніколи не спрацює → **головна сторінка залишиться без hreflang і без ін'єкту**;
- `redirect: false` — інакше `/files` отримає 301 на `/files/`, хоч це SPA-роут, а не директорія.

6.5 Структуровані дані (JSON-LD)

Схема	Де	Особливості
Organization	на кожній сторінці, включно з SPA-фолбеком	name, url, logo, description, foundingDate, address, contactPoint із мовами обслуговування
Product	сторінка товару з ціною	name, description, sku, mpn, image[] (усі фото), brand, category, offers
Offer	всередині Product	price у гривні, priceCurrency UAH, availability (In/OutOfStock), url, seller
BreadcrumbList	товар, категорія	Головна → Каталог → Категорія → Товар
Article	стаття блогу	headline (≤110), image, author/publisher = Organization, datePublished, dateModified
FAQPage	/faq і /en/faq	будується з таблиці faqs, локалізовано; порожня таблиця → схеми немає

Дві важливі деталі реалізації:

```
// 1. Escape '<' у JSON, щоб жодне поле не могло закрити <script> або створити тег
JSON.stringify(s).replace(/</g, '\\u003c')

// 2. Product без offers невалідний → для товарів без ціни віддаємо лише Breadcrumb+Organization
return hasPrice ? [product, buildBreadcrumb(crumbs), ORG_SCHEMA]
                  : [buildBreadcrumb(crumbs), ORG_SCHEMA]
```

⚠ **Габлі:** ціна в схемі мусить бути в тій самій валюті, що на сайті. Сайт показує EUR-товари в гривні за курсом НБУ. Якщо в схему віддати сире число EUR із priceCurrency: UAH, Google побачить ціну приблизно в 52 рази нижчу за реальну — з усіма наслідками для Merchant Center. Тому в server.js живе окреме дзеркало конвертера з тим самим коефіцієнтом (розділ 10.1).

6.6 Клієнтський SEO.jsx навмисно нічого не інжектить

```
// ⚠ ВАЖЛИВО — цей компонент НЕ інжектить жодних <meta>/<link>/<script>.
export default function SEO({ title }) {
  useEffect(() => {
    const full = /termojet/i.test(base) ? base : `${base} | Ternojet`
    if (full) document.title = full // ← змінює ТЕКСТ існуючого <title>
  }, [title, t])
  return null
}
```

Історія: тут стояв react-helmet, який дублював серверні теги (Googlebot виконує JS → у DOM 2 копії). Навіть після прибирання meta/OG/JSON-LD helmet створював другий елемент <title> замість заміни серверного.

✅ **Правило.** У моделі «сервер ін'єктить метадані» клієнт має право лише на document.title (щоб оновлювалась вкладка при SPA-навігації) і на нічого більше. Один тег — одне джерело.

⚠ **Граблі з подвоєнням бренду.** `seo_title` у БД вже брендovanі (... | Termojet), тому суфікс додається лише за відсутності бренду — інакше виходить «Termojet | Termojet».

6.7 Правила title і description (перевірені на 331 товарі)

Параметр	Правило	Чому саме так
<code>title</code> довжина	≤ 60 символів жорстко	понад 60 обрізається в SERP; у нас було 269 таких
<code>title</code> мінімум	≥ 35 символів	закороткі виглядають бідно; для коротких назв категорій код доклеює «— обладнання для котелень» до 60
<code>title</code> унікальність	100 %	дублі = <code>duplicate-title</code> ; у нас 11 пар довелося розводити руками
<code>title</code> структура	{Модель/Назва з розрізняльним атрибутом} \ Бренд	напр. «Колектор K42H.125(200) 4+1 вниз в теплоізоляції Termojet»
<code>description</code> довжина	120–160 символів	коротші за 120 аудит позначає як thin; довші обрізаються
<code>description</code> вміст	вигода + 1–2 факти + бренд	«Termojet — власне виробництво з 2002 року, доставка по Україні»
<code>description</code> унікальність	100 %	у нас стартувало з 361 дубля на весь сайт

Категорійні описи. Спочатку були 98–113 символів — аудит фіксував 111 закоротких. Подовжили `CATEGORY_META` до 120–155 символів для UA і EN.

EN-специфіка. Англійські переклади довші за українські: після перекладу 39 сторінок мали title > 60. Рішення — скрипт із правилами скорочення:

```
const RULES = [  
  [/Distribution Manifold/g, 'Manifold'],  
  [/Manifold with Hydraulic Separator/g, 'Manifold + Separator'],  
  [/ with Ball Valves/g, ', Ball Valves'],  
  [/ - Stainless Steel AISI 304/g, ''],  
  [/ Insulated/g, ''],  
  ...  
]  
// далі fit60(): обрізає по межі слова, гарантовано лишає ' | Termojet'
```

Було 61 задовгий → стало 0.

6.8 ⚠ Найдорожчі граблі проекту: ре-імпорт стирає SEO-поля

Дедуплікація `seo_title` робилась у **продовій БД** (файл у volume, не в git). Далі колега зробив ре-імпорт товарів — і **дублі повернулися**: імпорт перезаписав базовий `seo_title`.

Що вижило й що ні: - `i18n` (EN-скорочення) — здебільшого вижили; - UA `seo_title` — ні.

Рішення — ідемпотентний скрипт у git, який ганяється **після кожного** імпорту:


```
ssh hetzner && cd /home/tankoseva/termojet-website
cp data/termojet.db data/termojet.db.bak-$(date +%Y%m%d) # бекап спершу
docker compose cp scripts/seo-fix-titles.cjs app:/tmp/fix.cjs
docker compose exec app node /tmp/fix.cjs /app/backend/data/termojet.db # dry-run
docker compose exec app node /tmp/fix.cjs /app/backend/data/termojet.db --apply # застосувати
```

✅ **Правило (закласти в архітектуру, а не в пам'ять людини).** Будь-який імпорт товарів мусить **вбудовано** викликати постобробку SEO-полів. Варіанти: 1) імпорт не чіпає `seo_title` / `meta_description`, якщо вони непорожні (найкраще); 2) імпорт у кінці сам запускає нормалізацію title; 3) окремий крок деплою після імпорту. Покладатися на «не забути прогнати скрипт» не працює — ми забули.

🔄 **І це сталося вдруге.** Аудит 17 серпня знову показав **ті самі 5 груп дублів на 10 сторінках:** колектори K42H/K42B/K52H/K52B.125 у виконаннях (200)/(240) і НГ-52.150. Причина ідентична: 11 серпня в каталог залили нову партію товарів, а `seo-fix-titles.cjs` після цього не ганявся.

Висновок для нового магазину жорсткіший, ніж здавалося в липні: **правило, яке живе тільки в документації, порушується.** Захист має бути там, де його неможливо обійти, — у самому коді імпорту (варіант 1) або в CI/деплої (варіант 3).

6.9 Боротьба з soft-404: білий список роутів

Раніше catch-all був **чорним списком**: 404 віддавався лише для WordPress-патернів. Наслідок — будь-який неіснуючий URL (`/catalog/фейк` , `/випадкове`) віддавав **200 + порожню SPA-заглушку**, і Google індексував порожні сторінки.

Тепер — **білий список**:

```
const KNOWN_ROUTES = new Set(['/ ', '/catalog', '/blog', '/faq', '/about', '/service',
  '/delivery', '/oem', '/partners', '/returns', '/navchannya', '/portfolio',
  '/contacts', '/files', '/reviews', '/privacy', '/terms', '/cart', '/dealers',
  '/support', '/training', '/warranty'])

function isKnownRoute(p) {
  if (KNOWN_ROUTES.has(p)) return true
  if (p === '/admin' || p.startsWith('/admin/')) return true
  // /catalog:cat – категорія існує в довіднику?
  // /catalog:cat:slug – товар існує і is_visible=1?
  // fail-open на помилці БД: краще показати сторінку, ніж хибно 404-ити реальний товар
  // /blog:slug – стаття існує і published=1?
}
```

Додатково:

```
// Неіснуючі службові файли не маскуємо SPA-заглушкою
if (/\. (xml|txt|json|map|ico)$/i.test(req.path))
  return res.status(404).type('text/plain').send('Not found')

// Мертві WordPress-URL: користувач бачить сторінку 404, Google коректно деіндексує
const DEAD_WP = /^(^|\/)(product|product-category|author|feed|sample-page|comments|wp-
json|xmlrpc\.php)(\/|$)|^\/ru(\/|$)/i
```

✅ **Правила:** - 404 має бути **справжнім HTTP 404**, а не 200 з написом «не знайдено»; - `sitemap.xml`, `robots.txt` та інші службові файли, яких немає, віддають 404 текстом — інакше вони віддають HTML і ламають валідацію; - перевірка існування товару робить **fail-open** при збої БД: хибний 404 на реальному товарі гірший за зайвий 200.

6.10 Редиректи 301 зі старих URL

`backend/redirects.json` — 280 правил, генерується `scripts/gen-redirects.mjs` зі списку старих URL (витягнуто з `wp-sitemap.xml`). Стратегія зіставлення:

1. `/product/{slug}` → якщо slug існує → точна сторінка товару;
2. інакше → категорія за ключовим словом у слагі (мапа з ~15 евристик);
3. `/product-category/...` → відповідна категорія;
4. `/category/...` → `/blog`;
5. кореневі сторінки → мапа `PAGES` (`o-nas`, `kontakty`, `dostawa`, `gwarancja` ... — українські, польські й англійські варіанти);
6. решта → `/catalog` або `/blog` за натыком.

Далі по-ор правила (старий шлях = новий) видаляються.

⚠️ **Габлі з мовними префіксами.** Middleware стрипає `/pl /en /de /fr` і шукає UA-відповідник — **але тільки якщо стрипнутий шлях реально є в карті редиректів**. Інакше чинні `/en/catalog/...` отримували 301 і мультимовність ламалася.

```
if (lng) {
  const stripped = lng[2] || '/'
  if (REDIRECTS[stripped] && REDIRECTS[stripped] !== req.path)
    return res.redirect(301, REDIRECTS[stripped])
  return next() // ← чинний /en/... маршрут, пропускаємо
}
```

Ще одна деталь: шлях приходить URL-закодованим (кирилиця як `%D0%B0...`), тому перед пошуком у карті робиться `decodeURIComponent`.

6.11 Sitemap

`scripts/gen-sitemap.cjs` читає БД і генерує `public/sitemap.xml`: 768 URL (UA + EN пари) з `html:reflang` і `lastmod`.

```
статичні: 16 сторінок × 2 мови, priority 1.0...0.3, changefreq daily...yearly
категорії: унікальні category_slug видимих товарів × 2, weekly, 0.8
товари: is_visible=1 та є slug і категорія × 2, weekly, 0.7, lastmod = created_at
блог: published=1; EN-версія лише якщо є i18n.en.title, monthly, 0.6
```

⚠ **Габлі: у Docker-білдері немає БД.** І `backend/node_modules`, і `data/termojet.db` виключені через `.dockerignore`. Тому скрипт **не падає**, а акуратно виходить, лишаючи закомічений `sitemap.xml`:

```
try { Database = require('../backend/node_modules/better-sqlite3') }
catch { console.warn('...недоступний — лишаю наявний public/sitemap.xml'); process.exit(0) }
```

✅ **Правило.** Оновлення sitemap — **локальна операція + коміт**: `node scripts/gen-sitemap.cjs` → `git add public/sitemap.xml`. Інакше після деплою sitemap застигне на моменті останнього коміту.

💡 **Зробити краще.** Віддавайте sitemap **динамічно** з бекенду (`GET /sitemap.xml` прямо з БД, кеш 1 год). Тоді нові товари з'являються в ньому автоматично й крок «не забути згенерувати» зникає.

⚠ **І це вже спрацювало проти нас.** Станом на 17 серпня `public/sitemap.xml` датований **3 серпня** і містить 770 URL, а 11 серпня в каталог залили 12 нових товарів (фланцеві гідрострілки й сепаратори). Тобто **нових товарів у sitemap немає** — Google дізнається про них тільки через внутрішні лінки.

Дзеркальна проблема з того ж аудиту: **270 URL із sitemap не були закраулені** (`crawl/sitemap-coverage`) — переважно EN-сторінки, на які веде менше двох внутрішніх посилань. Sitemap обіцяє сторінки, яких немає в структурі лінків; краулер їх ігнорує. Разом із 235 orphan-сторінками це один симптом: **EN-версія слабо перелінкована зсередини**.

6.12 robots.txt

```
User-agent: *
Allow: /
Sitemap: https://termojet.com.ua/sitemap.xml
Disallow: /admin
```

✅ Мініمالізм навмисний: LLM-боти (GPTBot, PerplexityBot, ClaudeBot) **не блокуються**. Для B2B-магазину згадки в AI-відповідях — канал трафіку, і саме через це важливий SSR-блок з розділу 6.4: без нього боти без JS бачать порожню оболонку.

6.13 Фіди Google Shopping

`backend/routes/merchant.js` — RSS 2.0 з простором `g:`, 5 мов:

```
/google-merchant.xml      (uk)
/google-merchant-en.xml   /google-merchant-pl.xml
/google-merchant-de.xml   /google-merchant-fr.xml
```

Поля: `g:id` (sku або id), `g:title`, `g:description`, `g:link`, `g:image_link`, `g:availability`, `g:price` (завжди **UAH**), `g:condition=new`, `g:brand`, `g:mpn`, `g:google_product_category` (числові ID таксономії), `g:product_type`.

Правила відбору позиції у фід — товар пропускається, якщо немає slug, категорії, фото або ціни > 0. Кеш — `Cache-Control: public, max-age=3600`.

Числові ID таксономії Google, які ми використовуємо:

Категорії	ID	Гілка
групи, колектори, гідрострілки, BOX, Mega, сепаратори, тепла підлога	133	Hardware > Plumbing
насоси	500096	Hardware > Hardware Pumps
клапани, балансувальні	2466	Plumbing Valves
зональне керування	1897	HVAC Controls > Thermostats
автоматика	1519	HVAC Controls
додаткове, акція	632	Hardware

Ще одна дрібниця з практики: з опису для фіда вирізається службовий префікс (`^(Опис|Description|Opis|Beschreibung) [\s:—]+`) і робиться обрізка до 4900 символів (ліміт Google — 5000).

6.14 Що НЕ треба виправляти: артефакти «сирий vs відрендерений»

Аудитори будуть показувати попередження, які в нашій архітектурі — **хибні спрацювання**. Ми задокументували їх, щоб не витратити час повторно:

Попередження	Реальність
<code>landmark-one-main</code> , немає skip-link	<code><main id="main"></code> і skip-link є в <code>App.jsx</code> — у відрендереному DOM. Ін'єкт їх не додає в сирий HTML
<code>thin-content: 61 слово</code>	вважає лише сирий HTML статичних сторінок; тіло рендерить React
<code>orphan-pages: 349/491</code>	артефакт обмеження краулу × подвоєння на EN
немає лінка на privacy/contact	Footer існує в DOM; <code>/privacy</code> віддає HTTP 200
<code>csp unsafe-inline</code>	свідомо: GTM + інлайн-стилі анімацій. Прибрати можна лише через nonce/hash — високий ризик зламати аналітику, нуль користі для ранжування
<code>unminified-js</code>	евристика «є 4 коментарі»; Vite мініфікує
<code>http→https 301</code>	коректна поведінка
числові слагги (info)	спадок; виправлення = 301 + ризик колізій

✅ **Правило.** Перед тим як «виправляти» знахідку аудиту — перевірте живий вивід (`curl` для сирого HTML, headless-браузер для DOM). Половина попереджень на CSR-сайтах — про різницю між цими двома шарами, а не про дефект.

6.15 Зведений SEO-чекліст для нового магазину

- ☐ Кожен товар має заповнені `seo_title` (≤60, унікальний) і `meta_description` (120–160)
- ☐ Ці поля **редагуються з адмінки** з лічильником символів і прев'ю сніпета
- ☐ Метадані потрапляють у **сирий HTML** (SSR або ін'єкт) — перевірено `curl`

- `canonical` = self на кожній сторінці; **один** тег на сторінку
- `og:url` = `canonical` (інакше 331 попередження, як у нас)
- Категорії мають унікальні title/description і **видимий** текст-опис
- Статичні сторінки мають власні title/description (не generic)
- JSON-LD: Organization усюди, Product+Offer на товарах **з ціною**, BreadcrumbList, FAQPage, Article
- Ціна в схемі — у валюті відображення, після конвертації
- Неіснуючі URL → **HTTP 404** (білий список роутів, не чорний)
- Службові файли, яких немає → 404 текстом, не HTML
- Старі URL → 301 за картою; мовні префікси не ламаються
- sitemap.xml з `lastmod`, лише видимі товари й опубліковані статті; краще динамічний
- robots.txt: `Disallow: /admin`, посилання на sitemap, LLM-боти не заблоковані
- Внутрішня перелінковка в сирому HTML: breadcrumbs, схожі товари, навігація розділів
- Ре-імпорт товарів **не стирає** SEO-поля (перевірити явно!)
- Фіди Merchant для кожної мови, з правильною таксономією
- Після кожного релізу — прогін аудиту з фіксацією діфу оцінки

6.16 Розбір регресії 81 → 77: як магазин втрачає бали без жодної правки коду

Це найцінніший кейс документа, бо він показує **типовий життєвий цикл** магазину після запуску. Між зрізами 10 липня (81/B) і 17 серпня (77/C) у SEO-механіку **не внесли жодної зміни, що погіршує**.

Навпаки — додали SSR-блок `#seo-content` і виправили soft-404. Оцінка все одно впала. Ось повний розбір причин.

#	Просіло	Було → стало	Реальна причина
1	Images	100 → 86 (+1 error)	5 фото > 1 МБ (до 3.9 МБ), 111 фото > 200 КБ
2	Performance	94 → 85	сумарна вага ресурсів 63.9 МБ — прямий наслідок №1
3	Content	93 → 84	thin content на 82 сторінках + keyword stuffing
4	Core SEO	99 → 94	85 сторінок із задовгим/закоротким <code>meta_description</code>
5	дублі title	0 → 5 груп / 10 сторінок	заливка товарів 11 серпня стерла дедуп (розділ 6.8)
6	Accessibility errors	0 → 9	биті HTML-описи: <code></code> поза <code></code> , <code><tr></code> поза <code><table></code>

Причина №1 і №2: оптимізація зображень покривала лише один шлях. Правило `webp` у `nginx` (`location /wp-content/`) стосується **тільки** старих фото з WordPress. Нові фото товарів кладуться в `public/images/` — і для них **немає ні webp, ні стиснення**. Файли на 2.0–2.2 МБ у PNG лежали там місяцями: `sbv-klapan.png` (2.0 МБ), `skoba-45mm.png` (2.2 МБ), `truba-pert-16x2.png` (2.1 МБ), а `spe12-1.4.png` — **3.9 МБ**.

⚠ **Граблі.** Оптимізацію прив'язали до місця зберігання, а не до типу ресурсу. Поки всі фото були в одній теці — працювало; щойно з'явилась друга — тихо перестало, без жодного попередження. ✅ **Правило:** **стиснення й конверсія у webp/avif мають спрацьовувати на кожне завантажене фото в момент завантаження** (в `/api/upload`), а не постфактум правилом веб-сервера для конкретної теки. Плюс жорсткий ліміт: понад 300 КБ на фото товару адмінка має відхиляти або стискати автоматично.

Причина №3: SSR-блок наповнений не для всіх типів сторінок. `/catalog` віддає в сирому HTML 23 слова, `/en/catalog` — 21, сторінка категорії `avtomatyka` — 57 при мінімумі 300. Тобто головна сторінка каталогу — найважливіший вузол сайту — для краулера майже порожня: у неї є title і description, але немає тіла. Наповнення `#seo-content` зробили для товару, категорії, головної, блогу й FAQ, а `/catalog` лишився з одним абзацом.

Супутне: `keyword-stuffing` на `/en` — щільність слова «termojet» 39.6 %. Це артефакт того самого SSR-блоку: він перелічує назви категорій і товарів, у кожній з яких є бренд.  **Правило: у машинно-згенерованому SEO-тексті бренд згадується один раз у вступі, а не в кожному пункті списку.**

Причина №4: `meta_description` поза цільовим діапазоном на 85 сторінках. Два різні дефекти: - **закороткі** — статичні сторінки, зашиті в `STATIC_META`: `/service` 72 символи, `/contacts` 99, `/about` 119 (потрібно ≥ 120); - **задовгі** — англійські описи товарів: 170–182 символи (потрібно ≤ 160). Причина та сама, що з title: переклад довший за оригінал, а обрізання робили тільки для `seo_title`.

Причина №6: у контент потрапив невалідний HTML. На трьох сторінках (`84040tj-r-w-03`, `pump-grandlift-20-75sw`) в описі товару є `<li>` без обгортки `<ul>` і `<tr>` без `<table>`. Це не помилка рушія — це дефект **тексту в базі**, який рендериться через `dangerouslySetInnerHTML`.  **Правило: валідувати HTML опису при збереженні товару** (парсер + перевірка структури списків і таблиць), інакше биті теги живуть у продакшні місяцями.

Що НЕ просіло і чому це головний висновок. Structured Data, Social, Mobile, i18n, Local SEO, Analytics — рівно 100 у всіх трьох зрізах. Core SEO тримається на 94 навіть після заливки. Тобто **все, що зашите в код (ін'єкт метаданих, схеми, canonical, hreflang, 404), не деградує**. Деградує винятково те, що залежить від ручної дисципліни при наповненні: вага фото, довжина описів, валідність HTML, повторний прогін нормалізації title.

 **Головний висновок для нового магазину.** Проєктуйте не «правильний стан», а **механізм, який не дає з нього вийти**: валідація при збереженні товару, автоматичне стиснення фото, вбудована нормалізація SEO-полів після імпорту, динамічний sitemap і аудит у CI з падінням збірки при регресії оцінки.

7. Копірайтинг: методика, за якою написано 331 опис

Це готова, перевірена на всьому каталозі методика. Її можна брати як ТЗ копірайтеру нового магазину без змін.

7.1 Підготовка: ресерч-бриф на кожну категорію

Перед написанням описів для категорії робиться **ресерч-бриф** — окремий документ на 300–700 рядків. Вивчаються **офіційні сайти виробників країн-походження** (не магазинів-перекупів):

Категорія	Референси
групи, колектори, гідрострілки	Meibes/Flamco (DE), Oventrop (DE), PAW (DE), Barberi (IT), Caleffi (IT)
насоси	Wilo (DE), Grundfos (DK), DAB (IT), Lowara/IMP
клапани	Afriso, Danfoss, ESBE
сепаратори	Spirotech, Flamco, Caleffi, Reflex
тепла підлога	Danfoss, Herz, Rehau
зональне керування	Danfoss Icon, Salus, Tech, Computherm

Бриф містить: 1. **термінологію** галузі точними словами («гідравлічне розділення», «зона низьких втрат тиску», «первинний/вторинний контур», «підмішування зворотки», «міжосьова відстань 125 мм»); 2. **призначення й принцип роботи** кожного підтипу; 3. **критичні для покупця параметри** (що він насправді порівнює); 4. **сценарії застосування**; 5. **SEO-ключі трьома рівнями**: основні / другорядні / long-tail; 6. **типові питання** (FAQ); 7. **матрицю сумісності** — для перелінковки.

Приклад блоку ключів (насосні групи):

Основні:	насосна група · насосна група для котельні · насосна група з термостатичним краном · насосна група зі змішувачем
Другорядні:	насосна група купити · насосна група Termojet · насосна група для теплої підлоги · обв'язка котла
Long-tail:	насосна група 1 дюйм без насоса ціна · насосна група з термоголовкою для теплої підлоги · насосна група для твердопаливного котла

✅ **Правило.** Ресерч робиться **до** написання й фіксується у файлі. Інакше описи виходять або переспамленими, або з вигаданою технікою — а в опалювальному обладнанні хибна цифра означає підбір неправильного вузла на об'єкт.

7.2 Структура опису товару (перевірена, 300–450 слів)

#	Блок	Обсяг	Що містить
1	Вступ	40–60 слів	що це + головна вигода. Основний ключ у першому реченні. Модель і ключова характеристика

#	Блок	Обсяг	Що містить
2	Призначення та принцип роботи	60–90 слів	інженерно, але зрозуміло. 1–2 другорядні ключі
3	Сфери застосування	50–70 слів	конкретні контури й об'єкти. Тут природні внутрішні лінки
4	Переваги (булети)	50–70 слів	комплектація, матеріали, теплоізоляція, власне виробництво, гарантія
5	Технічні параметри	40–60 слів	ключові цифри з реальних specs
6	Переліноквка	вплетена	1 лінк на категорію + 1–2 на супутні товари + 1 зовнішній (де доречно)
7	Mini-FAQ	опційно, 2–3 питання	під featured snippets

Фактичний приклад із продакшну (початок опису НГ-46, [ng-46p-z-termokranom-1](#)):

Насосна група з термостатичним краном НГ-46 — попередньо зібраний вузол Termojet для контурів, що потребують точного підтримання температури теплоносія без автоматики керування котельнею. Триходовий термостатичний клапан утримує температуру подачі в діапазоні **25–55 °C**, а теплоізоляційний кожух зводить тепловтрати в котельні до мінімуму.

Принцип роботи побудований на підмішуванні зворотки: щойно температура подачі перевищує задану, термостатичний змішувальний клапан додає охолоджений теплоносій зі зворотної лінії [...] Вбудований зворотний клапан унеможливорює паразитну циркуляцію, а крани з термометрами 0–120 °C [...]

Типові сценарії застосування — тепла підлога, низькотемпературне радіаторне опалення [...] У котельні група монтується на **розподільчий колектор K22B.125 (2+1, виходи вгору)** зі стандартною міжосьовою відстанню 125 мм.

Переваги насосної групи Termojet: - готовий до монтажу вузол: запірні крани, термометри, зворотний клапан і змішувальний термокран уже зібрані й перевірені на виробництві; - теплоізоляційний кожух у комплекті [...] - українське виробництво: доступна ціна, наявність на складі та офіційна гарантія.

Технічні параметри: максимальна потужність контуру **15 кВт при $\Delta T=10\text{ °C}$** та **30 кВт при $\Delta T=20\text{ °C}$** , Kvs 3,5 м³/год, підключення 1" ВР, робочий тиск до 6 бар, габарити 353×261 мм, об'єм 0,34 л. Посадкова довжина насоса — 130 мм; насос у комплект не входить [...]

7.3 Жорсткі правила копірайтингу

✅ Правила, які не порушуються:

- Усі цифри — виключно з реальних **specs** товару.** Це вимога №1. Валідатор окремо прапорить будь-яке число з тексту, якого немає в характеристиках. Перевіряються: Qmax при $\Delta T=10/20$, Kvs, Gmax, DN, тиск, кількість контурів, міжосьова відстань, габарити.
- Основний ключ — у першому реченні**, далі 2–3 рази природно по тексту. Без переспаму.
- Тон — експертний**, без маркетингової «води». Стилль Meibes/Oventrop: інженерно-точний, з конкретними цифрами й термінами.

4. **Бренди конкурентів не називаються.** Лише натяки: «гідна заміна імпортованому», «той самий клас енергоефективності». Причина — і юридична, і репутаційна. Ми окремо вичищали згадки Danfoss/Caleffi/Giacomini зі списків адаптерів і TECH/eModul з описів автоматики (наш OEM).
5. **Не вигадувати параметрів, яких немає.** У насосах у specs є лише Hmax/Qmax/потужність/монтажна довжина/різьба — тому EEI, IP, ККД % **не пишемо для конкретного товару**, тільки як загальну властивість класу.
6. **Перелінковка — лише на реальні slug.** Валідатор парсить усі `/catalog/(cat)/(/slug)?` з тексту й звіряє з базою. Цільовий показник — 0 битих лінків (у пілоті було 33/33 валідні).
7. **Кожна цифра, що згадується в кількох описах, мусить збігатися.** Валідатор `check-nums.cjs` ловив крос-згадки: «СК-271 = 1135 мм», «проставки 220/250/280», «ГС-31 = 270 мм».

7.4 Три формати опису — і чому вибрали середній

Для узгодження з власником робились три варіанти на одному товарі (насос APE 25-60):

Формат	Обсяг	Склад
A	~343 слова	лаконічний: призначення + переваги
B 	~519 слів	обраний: повний опис + стисла таблиця параметрів
C	~1038 слів	максимальний: + докладна теорія й порівняння

Обрали **B**. Практичний висновок: понад ~550 слів для товарної сторінки не дає приросту — текст перестають читати, а ключі й так покриті.

Короткі описи (30–150 слів) для позицій, «про які нема чого писати» — на вимогу власника. Це муфти, переходи, євроконуси, скоби такера, плівка, кріплення. Для них формат B — штучна вода. У нас так описано ~40 позицій із 331.

 **Правило.** Формат вибирається за товаром, а не за шаблоном: складний вузол → повний опис; метиз → 2–3 речення по суті.

7.5 Перелінковка: карта, а не інтуїція

Для кожної категорії будується **link-map** — карта того, що з чим сумісне. Приклади реальних правил:

```

Насосна група — монтується на —> колектор із міжосьовою 125 мм
                — потребує —> насос відповідної посадкової довжини
Насос — за посадковою довжиною —> сумісні групи:
    1" + 130 мм → НГ-46, НГ-48
    1" + 180 мм → НГ-48, НГ-47 EPP
    1¼" + 180 мм → НГ-51.150, НГ-52.150
    фланцеві → серія Mega
Клапан — за таблицею сумісності —> електропривід (401N/405N/411/412N/413)
        і привід має ЗВОРОТНИЙ лінк на клапани
Тепла підлога: колектор ↔ змішувальний вузол ↔ шафа (за кількістю виходів)
                ↔ труба ↔ євроконус ↔ скоба ↔ плівка
Зональне керування: термостат ↔ центр комутації ↔ привід ↔ хаб ↔ датчик
                    (сумісність підтверджена PDF-інструкціями, не вгадана)

```

Плюс завжди: **1 лінк на категорію** каталогу і, де доречно, лінк на суміжний проєкт (у нас — сайт теплових насосів tjheatpump.com.ua для низькотемпературних систем).

⚠ **Граблі.** Перша ітерація насосів пропонувала 25-ті насоси до груп НГ-51/52 — неправильно за підключенням. Довелось регенерувати всі 44 описи. Причина: сумісність визначали «на око», а не за матрицею (підключення × монтажна довжина). ✅ **Правило: матриця сумісності складається до генерації описів і валідується.**

Ще одна реальна правка: у насосах **фактичне підключення** — це внутрішня різьба накидної гайки з комплекту, **на розмір менша** за зовнішню різьбу насоса: гайка 1½" → підключення 1" ВР; гайка 2" → 1¼" ВР. Спершу в описах стояли розміри гайки — покупці підбирали б неправильні фітинги.

7.6 HTML у полі опису

Опис зберігається як **HTML** і рендериться напряму:

```
// Новий формат — готовий HTML (<p>, <ul>, <a>): рендеримо як є (наш контент)
if (/<(p|ul|ol|h[1-6]|a|strong|br)\b/i.test(desc)) → dangerouslySetInnerHTML
// Старий формат (нумерована комплектація «N — ») лишився фолбеком
```

Чотири механізми навколо цього:

1. **plainText(desc)** — прибирає теги для мета-опису й фідів.
2. **SPA-навігація по внутрішніх лінках опису**: клік по `<a href="/catalog/...">` перехоплюється й виконується через `useNavigate` — без перезавантаження сторінки.
3. **localizeHtml** — переписує внутрішні `href` під мовний префікс, щоб на `/en/...` лінки в тексті вели на `/en/...`.
4. **Прибирання дубльованої таблиці**: частина описів містила вбудовану таблицю характеристик, яка дублювала канонічну секцію «02 · Технічні характеристики». Регулярка видаляє заголовки «... характеристик...» разом із наступною `<table>`, а також будь-яку решту `<table>`.

⚠ **Граблі.** `dangerouslySetInnerHTML` безпечний **лише** тому, що описи — наш власний контент, який заводиться скриптами. Якщо опис зможе редагувати зовнішній користувач — обов'язкова санітизація (DOMPurify).

⚠ **Другі граблі — невалідна структура.** Аудит 17 серпня знайшов у описах трьох товарів `<li>` **поза** `<ul>` і `<tr>` **поза** `<table>`. Для React це не помилка — він виведе що дадуть; для доступності це **9 помилок рівня error** (`ally/listitem`, `ally/list-structure`, `ally/aria-required-parent`). ✅ **Правило: валідувати структуру HTML при збереженні товару** — прогнати опис через парсер і відхилити, якщо елементи списку чи таблиці стоять без батьківського контейнера. Інакше биті теги живуть у продакшні місяцями й ніхто їх не бачить.

7.7 Конвеєр: як 331 опис доїхав до продакшну

1. 00-research-brief-<категорія>.md	← ресерч конкурентів і ключів
2. prep-<категорія>.cjs	← вибірка товарів + specs + link-map → батчі по ~7–15
3. генерація по батчах	← вихід: JSON {slug, name, seo_title, meta_description, h1, description_html, keywords, internal_links}
4. validate-<категорія>.cjs	← JSON валідний? лінки реальні? брендів немає?
check-nums.cjs	← кожна цифра є у specs?
5. assemble-all.cjs	← → backend/scripts/seo-descriptions-data.json [{slug, description}]
	→ backend/scripts/seo-meta-data.json [{slug, seoTitle, metaDescription}]
6. apply-seo-descriptions.js	← оновлює BCI місця даних (розділ 15.1)
7. білд + деплой + прогін apply у контейнері	
8. report-gen.cjs	← HTML-звіт по всіх описах для вчитки людиною

✓ Правила конвеєра, які себе виправдали:

- **Пілот перед масштабом.** Спочатку 1 товар на прод → показати → 10 товарів → узгодити формат → лише потім усі. Тричі це врятувало від масової переробки.
- **Валідація — обов'язковий, а не бажаний крок.** Цільові показники кожної партії: 0 битих лінків, 0 брендів конкурентів, 0 цифр поза specs.
- **Ідемпотентність apply-скрипта.** Повторний прогін безпечний — це дозволяє ганяти його після кожного деплою.
- **Батчі по 7–15 товарів.** Більші партії дають обриви й нестабільну якість.
- **HTML-звіт для вчитки.** Людина мусить побачити текст очима до продакшну; ми генерували `Termojet-SEO-описи-BCI-<N>.html` на кожному етапі.

7.8 FAQ як SEO-актив

21 реальне питання (8 про компанію + 13 технічних) заведено в таблицю `faqs` українською й англійською через ідемпотентний скрипт (upsert за текстом питання).

Технічні питання взяті з реальних дзвінків і листів: підбір насосних груп і колекторів, для чого гідрострілка, що таке КГС, ротаметри, сепаратори (шлам на зворотці / повітря на подачі, магнітні домішки), балансувальний клапан, BOX/Mega, сумісність, насоси APE/APM/APM-F, титановий анод.

Ці дані працюють двічі: сторінка `/faq` і `FAQPage` JSON-LD (розділ 6.5).

✓ **Правило.** FAQ пишеться з питань, які реально задають менеджерам. Вигадані питання не дають ні featured snippets, ні користі покупцеві.

7.9 Типові помилки копірайтингу (наш власний список)

Помилка	Наслідок
цифра, якої немає у specs	покупець підбирає неправильний вузол; втрата довіри
назви брендів конкурентів	юридичний і репутаційний ризик

Помилка	Наслідок
опис коротший за 150 слів на складному товарі	thin content, немає ключів
опис довший за ~600 слів	не читають, приросту немає
однакові описи у варіантів товару	<code>duplicate-description</code> по всьому сайту
лінк на неіснуючий slug	404 у тілі опису, злиті сигнали
ключ у першому реченні відсутній	втрата найсильнішої позиції
службовий префікс «Опис:» на початку	попадає у фіди Merchant як частина тексту
таблиця характеристик у тілі опису	дублює канонічну секцію
SEO-поля не заповнені, «згенеруємо потім»	«потім» = 269 задовгих title і 361 дубль

8. Структура блоків сторінок

Тут — порядок і призначення блоків, без вигляду. Порядок має і UX-, і SEO-сенси.

8.1 Головна сторінка

#	Блок	Призначення
1	Hero: відео-фон на весь екран + заголовок + СТА	позиціювання виробника за 3 секунди
2	Смуга статистики (overlay внизу відео)	роки на ринку, об'єкти, потужність — соціальний доказ
3	Смуга переваг / УТП	«власне виробництво», «готовність до монтажу», гарантія
4	Сітка категорій (з фото, описом і підкатегоріями)	головний навігаційний вузол і найцінніша перелінковка
5	Блок продукції / хіти	вхід у каталог із конкретики
6	Промо мобільного застосунку	окремий продукт екосистеми
7	Карусель реалізованих об'єктів	E-E-A-T: реальні котельні, потужність, локація
8	Фото виробництва (з адмінки, з лайтбоксом)	доказ власного виробництва
9	Відгуки (реальні з API + добивання шаблонними)	соціальний доказ
10	Великий текстовий блок про компанію	тіло тексту для ранжування головної
11	Блок партнерства / дилерам	другий тип конверсії, не лише продаж
12	Футер: каталог, розділи, контакти, privacy	перелінковка + Legal

✅ **Правило.** Сітка категорій — не «красива секція», а несуча конструкція: з неї йде найбільша частина внутрішньої ваги на категорії. У SSR-версії головної ми віддаємо **всі 15 категорій із описами** саме тому.

8.2 Сторінка категорії (каталог)

#	Блок	Деталі
1	Шапка сторінки	H1 = назва категорії
2	Читабельний опис категорії	абзац під шапкою. Лікує thin content і дає on-page текст. Джерело — <code>categories.js</code>
3	Смуга плиток категорій	горизонтальна прокрутка + «Всі категорії»: перехід між категоріями без повернення назад
4	Сайдбар фільтрів	ціновий діапазон + групи фільтрів зі <code>specs</code> цієї категорії
5	Тулбар	пошук у категорії, фільтри (моб.), перемикач вигляду сітка/список
6	Сітка товарів	картка: фото, назва, артикул, ціна (з акційною), наявність, 2–3 бейджі ключових характеристик
7	Пагінація / довантаження	ліміт 100 на запит

Бейджі характеристик у картці — недооцінена деталь. Вони підбираються рег-категорія за пріоритетним списком ключів: для колектора — кількість виходів, для насоса — напір і продуктивність, для сепаратора — тип (повітря/бруд/комбі) і DN. Покупець порівнює позиції **не відкриваючи** товар.

⚠ Граблі мобільної версії. Плитки категорій відкривалися **з другого тапу**: CSS-hover-стани на тач-екрані перехоплювали перший тап. Фікс — гейтинг через `canHover()`. Перевіряйте цей сценарій на реальному телефоні, а не в емуляторі.

8.3 Сторінка товару (PDP) — найважливіша сторінка магазину

#	Блок	Деталі
1	Breadcrumb-смуга	Головна / Каталог / Категорія / Товар — і в DOM, і в JSON-LD
2	Галерея (ліва колонка)	вертикальні мініатюри + велике фото, зум, лайтбокс, стрілки, лічильник; окремий стан для мобільної
3	3D-модель (де є)	<code><model-viewer></code> , лениве завантаження після <code>whenDefined</code> ; badge «3D»
4	Метадані категорії	лінк на категорію (десктоп)
5	Trust-акордеони під галереєю	доставка · оплата · гарантія · виробництво — знімають типові заперечення там, де вони виникають
6	Панель купівлі (права колонка)	бейдж «власне виробництво», назва (H1), артикул, наявність, ціна (акційна + перекреслена), кількість, «Купити», «Консультація»
7	Короткий опис	1–2 речення суті
8	Чіпи ключових характеристик	до 5 параметрів (без «Артикул») — швидке порівняння
9	Документація товару	PDF-інструкції за категорією + за моделлю (мапа <code>docsMapping</code>)
10	02 · Технічні характеристики	канонічна таблиця зі <code>specs</code> (локалізовані ключі)
11	03 · Опис	повний HTML-опис 300–450 слів (розділ 7)
12	05 · Відгуки про товар	список + форма з фото; премодерація
13	04 · Схожі товари	4 позиції категорії — перелінковка

Нумерація секцій (02, 03, 04, 05) — не декор: вона задає незмінний порядок і дає змісту сторінки читабельну сітку. Порядок «характеристики → опис → відгуки → схожі» обраний свідомо: покупець технічного товару спершу перевіряє **цифри**, і лише потім читає текст.

Документація за моделлю — окрема тонкість. Мапа має два рівні: за категорією (`nasosni-hrupy` → інструкції НГ-47/48 і НГ-51/52) і за **збігом у назві** (`/APM-F/` → власна інструкція; `/\bAPM\b(?!-F)/` → інша). Негативний lookahead тут обов'язковий, інакше APM-F отримує інструкцію різьбових APM.

8.4 Обов'язковий набір статичних сторінок

Роут	Сторінка	Навіщо
<code>/about</code>	Про компанію	E-E-A-T: виробництво, історія, цикл, виставки
<code>/contacts</code>	Контакти	E-E-A-T + локальне SEO; аудит окремо перевіряє наявність

Роут	Сторінка	Навіщо
<code>/delivery</code>	Доставка й оплата	знімає заперечення до кошика
<code>/service</code>	Сервіс і гарантія	+ редиректи <code>/warranty</code> , <code>/support</code> сюди
<code>/returns</code>	Повернення й обмін	Legal-вимога
<code>/privacy</code>	Політика конфіденційності	Legal-вимога ; без неї аудит ставить Legal 44/100
<code>/terms</code>	Умови використання	Legal
<code>/faq</code>	Часті питання	+ FAQPage schema
<code>/files</code>	Документація й каталоги	сильна сторінка для B2B-трафіку
<code>/portfolio</code>	Реалізовані об'єкти	найсильніший доказ компетенції
<code>/reviews</code>	Відгуки	соціальний доказ
<code>/partners</code>	Дилерам і партнерам	B2B-конверсія (+ редирект <code>/dealers</code>)
<code>/oem</code>	ОЕМ-виробництво	окремий B2B-напрямок
<code>/navchannya</code>	Навчання	контент-маркетинг (+ редирект <code>/training</code>)
<code>/blog</code> , <code>/blog/:slug</code>	Блог	інформаційний трафік
<code>/cart</code>	Кошик	—

✅ **Правило.** Legal-сторінки (privacy, terms, returns) роблять на старті, а не «потім»: вони дешеві й прямо впливають на оцінку довіри до магазину. Історичні URL закриваються **редиректами всередині SPA** (`/warranty` → `/service`), а не 404.

8.5 Наскрізні елементи

- **Навбар** — прозорий над hero, перемикач мов, пошук, кошик із лічильником;
- **Skip-link** «Перейти до основного контенту» + `<main id="main">` — a11y;
- **Футер** — каталог, розділи, контакти, privacy (перелінковка + Legal);
- **Плаваючі кнопки** — швидкий контакт;
- **Мобільна нижня навігація** — головні розділи в зоні великого пальця;
- **Popup навчання** — лідмагніт;
- **ErrorBoundary** — падіння компонента не забирає всю сторінку;
- **ScrollToTop** — скрол на початок і **перезахоплення UTM** при кожній навігації.

9. Мультимовність

Сайт працює п'ятьма мовами: **uk** (канонічна) · **en** · **pl** · **fr** · **de**.

9.1 Модель зберігання

```
Українська → у власних колонках таблиці (name, description, short_desc, seo_title, ...)
Переклади → у JSON-колонці i18n тієї ж таблиці:
    { "en": { name, short_desc, description, specs, seo_title, meta_description },
      "pl": {...}, "fr": {...}, "de": {...} }
```

Колонка `i18n` є в `products`, `blog_posts`, `portfolio`, `reviews`, `faqs`.

✅ **Чому так, а не окремою таблицею перекладів.** Одна вибірка віддає товар з усіма мовами; немає JOIN; додавання мови — це запис у JSON, без міграції схеми. Мінус — не можна фільтрувати SQL-ом по перекладеному полю. Для каталогу до кількох тисяч SKU плюси переважають.

9.2 Віддача перекладів у API

`withI18n()` розкладає JSON у **пласкі поля** `<база>_<мова>`, які очікує фронтенд:

```
// { name: 'name', desc: 'description', shortDesc: 'short_desc',
//   specs: 'specs', features: 'features', subcategory: 'subcategory',
//   seoTitle: 'seo_title', metaDescription: 'meta_description' }
→ p.name_en, p.desc_pl, p.specs_de, p.seoTitle_fr ...
```

Порожні значення не додаються — компонент бере базове (українське) поле.

⚠️ **Габлі з локалізованими характеристиками.** `specs_<lang>` містить **перекладені ключі**. Тому логіка вибору ключа працює по канонічному українському `specs`, а відображення бере значення з `specs_<lang>` **за позицією** канонічного ключа. Якщо порядок ключів у перекладі зіб'ється — поїдуть значення. ✅ **Правило: у новому проєкті перекладайте лише значення й підписи, а ключ тримайте машинним** (`power_dt10`, `connection`). Тоді позиційне зіставлення не потрібне взагалі.

9.3 Роутинг

```
// UA — без префікса: /catalog/...
// Мовні дерева з одного шаблону:
[['en', 'pl', 'fr', 'de'].map(langRoutes)]
```

⚠️ **Габлі.** Спочатку існувало тільки дерево `/en`, а перемикач мов показував усі чотири → `/pl` вело на неіснуючий роут = **білий екран**. Тепер дерево генерується функцією для кожної мови — шаблон один.

Три допоміжні механізми: - `LangSync` — синхронізує мову в стані застосунку з префіксом URL; - `LLink` / `localizedPath` — компонент і хелпер: усі внутрішні посилання автоматично отримують мовний префікс. Є `guard` для зовнішніх URL; - `localizeHtml` — переписує `href` **усередині HTML-контенту** описів і статей.

⚠ **Габлі.** «Голі» `<Link>` замість `LLink` у breadcrumbs, блозі, портфоліо й блоці схожих товарів викидали користувача з англійської версії на українську. Знайшли 9 таких місць. ✅ **Правило:** заборонити прямий `<Link>` на публічних сторінках лінтером — тільки `LLink`.

9.4 Серверні метадані по мовах — асиметрія

Мова	Метадані в сирому HTML	Статус
uk	повністю	канонічна
en	повністю: <code>pickLang</code> + <code>STATIC_META_EN</code> + <code>hreflang</code>	пілот доведено
pl, fr, de	інтерфейс перекладений, серверного ін'єкту немає	дані <code>i18n</code> готові, механізм той самий

`hreflang` віддається парою `uk ↔ en` + `x-default` → `uk`.

✅ **Правило.** Розгортайте мови **по одній до кінця**: інтерфейс → контент → серверні метадані → `hreflang` → `sitemap` → Search Console. Пів-увімкнена мова створює дублі й порожні сторінки в індексі.

9.5 Конвеєр перекладів

- `scripts/i18n-merge.mjs` — deep-merge нових ключів у `translations.js`. Приймає `{ "namespace": { "key": { "uk": "...", "en": "...", "pl": "...", "fr": "...", "de": ..." } } }`, розгортає крапкові ключі (`a.b.c`) у вкладені об'єкти, зберігає файл наново.
- `backend/scripts/translate-content.js` — переклад контенту в колонку `i18n`.
- `patch-i18n-*.js` — точкові патчі перекладів для конкретних партій товарів.

⚠ **Габлі.** Частина англійських текстів у нас **витягнута машинно**, а не перекладена людиною — це видно в довжині title (звідси скрипт скорочення з розділу 6.7). Для комерційних мов (де реально йде трафік) переклад мусить вичитувати людина.

10. Комерція: ціни, кошик, замовлення, доставка

10.1 Дві валюти й курс НБУ

Частина асортименту закупается в євро, тому кожен товар має **власну валюту** (`currency` : `UAH` або `EUR`). Сайт **завжди показує гривню**:

```
const NBU_API = 'https://bank.gov.ua/NBUStatService/v1/statdirectory/exchange?valcode=EUR&json'
const MARKUP = 1.022 // курс НБУ + 2.2%
const CACHE_TTL = 3600_000 // кеш 1 година

toUAH(price, 'UAH', rate) → price
toUAH(price, 'EUR', rate) → Math.round(price * rate)
```

Фолбек, якщо НБУ недоступний: останній кешований курс, інакше `51 × 1.022`.

⚠ **Граблі: конвертер існує в чотирьох місцях.**

Копія	Файл	Для чого
1	<code>src/utils/currency.js</code>	відображення ціни на сайті
2	<code>backend/currency.js</code>	перерахунок суми замовлення
3	<code>backend/server.js</code> → <code>getEurRate()</code>	ціна в <code>Product</code> JSON-LD і в SSR-блоці
4	<code>backend/routes/merchant.js</code> → <code>eurRate()</code>	<code>g:price</code> у фідах Google

Коефіцієнт `1.022` і фолбек продубльовані **чотири рази**. Розсинхронізація дасть різну ціну на сторінці, у схемі, у фіді й у замовленні — найгірший клас розбіжностей для магазину.

✅ **Правило для нового магазину.** Один модуль курсу, який використовують усі споживачі (для CJS/ESM-розділення — спільний файл на CommonJS або build-step). Ще краще — **зберігати ціну в гривні в БД** і перераховувати її під час імпорту/оновлення прайсу, а не на кожному запиті. Тоді курс потрібен лише в одному місці — в імпортері.

Форматування цін локалізоване: `uk` → «7 945 грн», `en` → «UAH 7,945», `pl/fr/de` — локальний формат UAH через `Intl.NumberFormat`.

10.2 Ціна «по запити» та акційна ціна

- `price = 0` → замість ціни «ціна по запити»; товар **не потрапляє у фіди Merchant** і віддає `Product` -схему без `offers` (розділ 6.5);
- `sale_price > 0` і `< price` → показується акційна + перекреслена стара, товар автоматично попадає в категорію «Акція».

10.3 Кошик

Кошик живе у `localStorage` (`tj2_cart`) — без серверної сесії:

```
{ id, name, price, currency, image, quantity, ... } // ← поле quantity!
```

⚠ Пастка `quantity` vs `qty` описана в розділі 3.4 — це та сама помилка, яка зіпсувала суми в Telegram і листах.

10.4 Оформлення замовлення

Поля форми: ім'я, телефон, e-mail, спосіб оплати, коментар + **доставка Новою Поштою** (місто + відділення) або адреса.

Що відбувається на `POST /api/orders`:

1. Сервер перераховує `total` із БД (клієнтським цінам не вірить)
2. `INSERT` у `orders`: склад, сума, контакти, оплата, UTM, місто/відділення НП (+ Ref-и)
3. E-mail із замовлення → таблиця `subscribers` (`INSERT OR IGNORE` — дедуп за `UNIQUE`)
4. Telegram: картка замовлення в топик «🔵 Термоjet»
5. CRM: лід у `heat-pump-registration` із `source` і повним складом
6. `Resend`: лист-підтвердження клієнту (`fire-and-forget`)
7. Відповідь клієнту: `{ id }`
8. Фронт: GA4-подія `purchase` у `dataLayer`

✅ **Правило.** Кроки 4–6 — **fire-and-forget**: жоден зовнішній сервіс не може зламати оформлення замовлення. Замовлення вже в БД; недоступність Telegram, CRM або пошти лише позбавляє сповіщення, а не втрачає гроші.

10.5 Нова Пошта

Проксі на бекенді (ключ **ніколи** не йде у фронт):

```
GET /api/np/cities?q=київ      → [{ ref, label }] (searchSettlements, Limit 20, q ≥ 2 симв.)
GET /api/np/warehouses?cityRef=... → [{ ref, label, number }] (getWarehouses, Limit 500)
```

- кеш відділень по місту — **6 годин** (списки великі й майже не змінюються);
- таймаут запиту 8 с через `AbortController`;
- **немає ключа** → **503**, і кошик просто не показує підказок (форма не ламається);
- у замовленні зберігаються і назви, і `Ref` -и — щоб потім можна було створити ТТН програмно.

10.6 Аналітика електронної торгівлі (GA4 через GTM)

Контейнер GTM підключений у `index.html` (+ `<noscript>`-iframe). Події надсилаються у `dataLayer` за суворим форматом:

```
window.dataLayer.push({ ecommerce: null }) // ← скидання попереднього стану
window.dataLayer.push({ event, ecommerce })
```

Подія	Коли
<code>view_item</code>	відкриття сторінки товару
<code>add_to_cart</code>	додавання в кошик
<code>remove_from_cart</code>	видалення з кошика

Подія	Коли
<code>begin_checkout</code>	перехід до оформлення
<code>purchase</code>	успішне замовлення (<code>transaction_id</code> , <code>value</code> , <code>currency: UAH</code> , <code>items[]</code>)

Структура `item` : `item_name` , `item_id` (sku або id), `price` , `item_brand` , `item_category` (назва категорії, не slug), `item_category2` (підкатегорія), `quantity` .

✅ **Правило.** `ecommerce: null` перед кожною подією обов'язковий — інакше GTM змішує дані попередньої події з наступною й у звітах з'являються товари-привиди.

10.7 Власна UTM-атрибуція

Незалежно від GA4 ми зберігаємо UTM у своїй БД:

```
// captureUTM() при кожній навігації → sessionStorage 'tj_utm'
const UTM_KEYS = ['utm_source','utm_medium','utm_campaign','utm_term','utm_content']
// далі UTM додається до кожної заявки: orders.utm, consultations.utm, dealers.utm
```

`GET /api/analytics` віддає адміні: підсумки (замовлення / консультації / дилери / підписники / ліди / дохід), ліди по днях за 30 днів, топ-10 source / medium / campaign, розподіл замовлень за статусом.

✅ **Правило.** Власна атрибуція лідів у БД цінніша за звіти GA4: вона зв'язує конкретну заявку з каналом і не залежить від cookie-згод і блокувальників.

11. Інтеграції: повний реєстр

Спільний принцип **усіх** інтеграцій:

```
// Fire-and-forget: ніколи не блокує й не ламає HTTP-запит, який його викликав.  
function notifyX(payload) { sendX(payload).catch(() => {}) }
```

11.1 Telegram — сповіщення про заявки

- один бот на два сайти; спільна група-форум із **топіками**: заявки форм → топик « Termojet»; дзвінки → топик «TJ Heat Pumps»;
- `parse_mode: HTML` + **обов'язкове екранування** користувацького тексту (`&`, `<`, `>`) — інакше лапки в коментарі клієнта ламають повідомлення;
- `disable_web_page_preview: true`;
- ID топика має **хардкод-фолбек** (`169`), щоб при втраті ENV-змінної ліди не падали в General;
- немає токена або `chat_id` → сповіщення **тихо вимкнені**, без падінь.

11.2 CRM — пересилання лідів

`POST https://crm.tjheatpump.com.ua/api/leads`, тіло: `{ type, name, phone, email, message, source }`, де `type` $\in$ `consultation | partnership | order | call | other`.

`source` формується як `termojet.com.ua · Магазин · <utm_source>` — у CRM одразу видно канал. У `message` — склад кошика, доставка, оплата, коментар, сума, UTM.

Захист: таймаут 8 с (`AbortController`), опційний спільний секрет у заголовку `X-Lead-Secret`, помилка ковтається.

⚠ **Граблі: rate-limit на боці CRM — 5 запитів/хв на IP.** Усі ліди сайту йдуть з **одного** IP сервера. Сплеск під рекламну кампанію (6+ заявок/хв) впирається в `429` — заявка лишається в БД сайту й у Telegram, але в CRM не потрапляє. ✅ **Правило: при інтеграції сервер-до-сервера завжди звіряйте ліміт приймача й або додайте IP у виняток, або поставте чергу з повторами.**

11.3 Binotel — вхідні дзвінки як ліди

Вебхук `POST /api/webhooks/binotel` (метод API PUSH; налаштовує підтримка оператора на своєму боці).

Послідовність подій і обробка:

Подія	Що робимо
<code>receivedTheCall</code>	миттєве повідомлення «дзвонить зараз» у Telegram
<code>answeredTheCall</code>	ігноруємо
<code>hangupTheCall</code> (плаский, без <code>companyID</code>)	ігноруємо
<code>apiCallCompleted</code>	створюємо лід + повну картку: <code>billsec > 0</code> → прийнятий, інакше ПРОПУЩЕНИЙ ; <code>callType=1</code> (вихідні) → ігноруємо

У картці/ліді: ім'я клієнта, сторінка, з якої дзвонили, час на сайті до дзвінка, перший візит, UTM, гео, IP, GA client ID, ID дзвінка, посилання на запис розмови, хто відповів. Пропущений дзвінок у CRM автоматично стає задачею з високим пріоритетом.

Безпека: оператор запиту не підписує, тому — **IP-allowlist** + звірка **companyID**. Працює **лише** завдяки правильному **trust proxy** (розділ 3.3).

Діагностика без фейкових лідів:

```
docker logs -f termojet-website-app-1 | grep binotel
# [binotel] <requestType> from <ip> ext=... disp=... bill=...

# чужий IP → 403 forbidden; IP оператора + чужа компанія → 403 wrong company;
# вихідний (callType:1) → 200 без ліда
curl -H 'X-Forwarded-For: <IP оператора>' http://127.0.0.1:8080/api/webhooks/binotel -d '...'
```

11.4 Resend — лист-підтвердження клієнту

- HTML-лист на табличній верстці (сумісність із поштовими клієнтами), брендований;
- **from** — на **верифікованому домені** (SPF/DKIM у DNS обов'язково, інакше спам);
- **reply_to** — робоча скринька магазину з **settings** (відповідь клієнта йде людям);
- **bcc** — прихована копія менеджеру (опційно);
- склад замовлення, сума, доставка, оплата, контакти з **settings** ;
- **немає ключа** → **лист не йде**, замовлення це не блокує.

11.5 Зведена таблиця інтеграцій

Інтеграція	Напря́м	Критичність	Поведінка при збої
Telegram	вихідний	середня	тихо не надсилає
CRM	вихідний	середня	ковтає помилку, лід у БД
Resend	вихідний	низька	лист не йде
Нова Пошта	вихідний, синхронний	середня	503, форма без підказок
НБУ (курс)	вихідний	висока	кеш → фолбек 51
Binotel	вхідний	середня	403/ігнор
GTM/GA4	клієнтський	низька	немає даних
Merchant-фіди	вихідний (pull)	середня	Google бачить старий фід

12. Безпека

12.1 Заголовки й CSP

```
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: ...
```

CSP навмисно **поблажлива, але не пуста**:

```
default-src 'self' https: data: blob:
script-src 'self' 'unsafe-inline' 'wasm-unsafe-eval' https:
style-src 'self' 'unsafe-inline' https:
img-src 'self' data: blob: https:
font-src 'self' https: data:
connect-src 'self' https:
frame-src 'self' https:
worker-src 'self' blob:
object-src 'none'
base-uri 'self'
frame-ancestors 'self'
```

Що вона реально дає: блокує `http/mixed-content`, `inline-<object> / <embed>`, підміну `base-uri` і вбудовування сайту в чужі фрейми. Не ламає Google Fonts, GTM, мапи, запит курсу НБУ й 3D-в'юер.

Два свідомі рішення: - `wasm-unsafe-eval` **замість** `unsafe-eval` — потрібен WebAssembly для 3D-в'юера, але загальний `eval` заборонений. Це значно безпечніше; - `unsafe-inline` **лишено** — GTM і інлайн-стилі анімацій. Прибрати можна лише через nonce/hash: високий ризик зламати аналітику й анімації, нульова користь для ранжування.

12.2 CORS

```
const corsAllowed = process.env.CORS_ORIGINS?.split(',') ?? [
  'https://termojet.com.ua', 'https://www.termojet.com.ua',
  'https://app.termojet.com.ua', 'http://49.13.154.30:8080', 'http://localhost:5173',
]
app.use(cors({ origin: (o, cb) => cb(null, !o || corsAllowed.includes(o)), credentials: true })))
```

✅ **Правило: білий список, не `origin: '*'`**. Запити без Origin (сервер-до-сервера, curl) пропускаються.

12.3 Зведений чекліст безпеки

Пункт	Як зроблено
Секрет JWT	fail fast при <code>< 32</code> символів

Пункт	Як зроблено
Термін токена	24 год
Пароль адміна	bcrypt (12 раундів) із міграцією з плейнтексту при вході
Токен у браузері	<code>sessionStorage</code> , не <code>localStorage</code>
Брутфорс логіна	10 спроб / 15 хв
Публічні POST	20/хв
Завантаження	25 МБ, білий список розширень, без SVG , згенеровані імена
Секрети	тільки ENV ; <code>settings</code> віддається публічно
Ціни замовлення	перерахунок на сервері
Помилки	стектрейси лише в логи
Вебхук	IP-allowlist + звірка ID компанії
Реальний IP	<code>trust proxy: ['loopback','uniquelocal']</code>
Адмінка в robots	<code>Disallow: /admin</code>
Порт контейнера	<code>127.0.0.1:8080:3000</code> — не <code>0.0.0.0</code>
TLS	Let's Encrypt, автопродовження nginx-плагіном
JSON-LD	escape <code><</code> → <code><</code>
HTML користувача	екранування у Telegram і листах

⚠ **Граблі: публікація порту.** У compose порт біндиться **лише на localhost**:

```
ports:
  - "127.0.0.1:8080:3000" # без 127.0.0.1 порт стирчав би в інтернет в обхід nginx
```

Без цього префікса Docker відкриває порт на `0.0.0.0` , і сайт стає доступним напямую — **в обхід TLS, rate-limit і заголовків nginx**.

13. Продуктивність

13.1 Кешування статики

```
// /assets/* (Vite кладе контент-хеш у назву)
Cache-Control: public, max-age=31536000, immutable      // рік
// media з public/: png|jpg|webp|avif|svg|woff2|mp4|glb...
Cache-Control: public, max-age=604800                  // 7 днів
// index.html і кореневі файли
Cache-Control: no-cache                                // завжди свіжі
```

⚠ **Габлі, які було видно оком.** Медіа з `public/` віддавались із `no-cache` → браузер ревалідував кожне фото при кожній навігації: білий кадр, блимання картинок, зайвий трафік на важких PNG. Виправлення одним рядком дало найпомітніший для користувача приріст швидкості за весь проєкт.

13.2 Розмір JavaScript

Стартово — один бандл на **803 КБ**, `perf/js-file-size: error`.

Що зробили (–62 %): 1. головна сторінка eager, **решта 20+ сторінок через `lazy()` + `Suspense`**; 2. `manualChunks` для вендорів; 3. **найголовніше**: глобальний контекст більше не тягне 2 МБ статичного каталогу товарів — статика довантажується динамічним `import()` лише у фолбек-режимі.

Результат: найбільший чанк 2208 → 43 КБ, initial JS ~3.6 → 1.4 МБ.

✅ **Правило.** Перед оптимізацією подивіться, **що саме** лежить у бандлі. У нас головним «жирком» був не React і не бібліотеки, а дата-файл каталогу, який імпортувався статично «про запас».

13.3 Зображення

- **gzip** на всі відповіді (`compression()`) — ~75 % ваги HTML/JS/JSON;
- **webp через nginx** без зміни коду:

```
map $http_accept $webp_suffix { default ""; "~*image/webp" ".webp"; }
location /wp-content/ {
    try_files $uri$webp_suffix $uri @wp_old;
    add_header Vary Accept;
}
```

`.webp` лежить поряд із оригіналом (адитивно, оригінали не чіпаються). Реально: 308 КБ JPEG → 161 КБ webp (–48 %), фолбек безпечний.

Регенерація після додавання нових фото:

```
find /var/www/.../uploads -type f \( -iname '*.jpg' -o -iname '*.png' \) ! -name '*.webp' \
| while read f; do [ -f "$f.webp" ] || cwebp -quiet -q 82 "$f" -o "$f.webp"; done
```

⚠ **Габлі.** `width / height` на зображеннях так і не додані → CLS страдає. ✅ **Правило:** у новому магазині атрибути розмірів обов'язкові з першого дня — це дешево на старті й дорого потім.

⚠ ⚠ **Габлі, які коштували 14 балів Images і 9 балів Performance.** Уся ця webp-оптимізація прив'язана до **однієї теки** — `location /wp-content/`. Нові фото товарів лежать у `public/images/`, і для них не працює **ні webp, ні стиснення**. Аудит 17 серпня: **0 файлів .webp** у `public/images/`, 5 фото понад 1 МБ (найважче — 3.9 МБ), 111 фото понад 200 КБ, сумарна вага ресурсів сайту **63.9 МБ**. Найважчі — звичайні PNG товарів по 2.0–2.2 МБ, які там лежать із червня.

✅ **Правило:** оптимізація має бути прив'язана до **події завантаження, а не до шляху**. У новому магазині обробляйте фото в момент прийому в `/api/upload`: ресайз під максимальний потрібний розмір, конверсія у webp/avif, жорсткий ліміт ваги (наприклад, 300 КБ на фото товару). Тоді жодна нова тека не зможе «випасти» з оптимізації, а адмінка фізично не прийме 4-мегабайтний PNG.

13.4 3D-моделі

- прекомпресія: якщо поряд лежить `<file>.gz`, віддається він із `Content-Encoding: gzip` — **нуль CPU на запит**, браузер розпакує прозоро;
- важкі STEP зберігаються **лише** як `.gz` (економія диску й деплою ~4×);
- захист від виходу за межі каталогу: `gzPath.startsWith(UPLOADS_DIR + path.sep)` ;
- `<model-viewer>` вантажиться ліниво, після `whenDefined` .

13.5 Шрифти

`preconnect` до Google Fonts + `display=swap` + `subset=cyrillic` .

⚠ **Габлі.** Google Fonts у критичному ланцюжку — залишкове зауваження аудиту. Самохостинг шрифтів (`.woff2` у `public/` + `preload`) прибирає зовнішній домен із критичного шляху.

14. Деплой і оточення

14.1 Схема

Домен (реєстратор, DNS: A @ + A www) → Hetzner VPS

nginx :443 (Let's Encrypt, автопродовження)

domain → proxy_pass http://127.0.0.1:8080

www.domain → 301 на без-www (канонічний домен)

:80 → 301 на https

/wp-content/ → локальне дзеркало старих фото + webp

Docker Compose: один контейнер

Express :3000 → роздає dist/ + /api + /uploads

volumes: ./data → /app/backend/data (SQLite)

./uploads → /app/backend/uploads (файли)

nginx тут — **чистий проксі**: gzip, заголовки безпеки, 301-редиректи зі старих URL і роздачу 3D робить Express усередині контейнера. Так логіка не роздвоюється.

14.2 Реєстр ENV-змінних

Змінна	Обов'язкова	Призначення
JWT_SECRET	так (≥32)	підпис адмін-токена; без неї контейнер не стартує
PORT	ні (3000)	порт Express
NODE_ENV	ні	production
CORS_ORIGINS	ні	білий список origin через кому
TELEGRAM_BOT_TOKEN	ні	без неї сповіщення вимкнені
TELEGRAM_CHAT_ID	ні	ID групи (-100...)
TELEGRAM_THREAD_ID	ні	топік для заявок форм
TELEGRAM_CALLS_THREAD_ID	ні	топік для дзвінків
NOVA_POSHTA_API_KEY	ні	без неї /api/np → 503
CRM_LEADS_URL	ні	ендпоінт лідів CRM
CRM_LEAD_SECRET	ні	заголовок X-Lead-Secret
BINOTEL_COMPANY_ID	ні	звірка вхідних вебхуків
BINOTEL_ALLOWED_IPS	ні	IP-allowlist (CSV)
BINOTEL_API_KEY / _SECRET	ні	REST оператора (записи розмов)
RESEND_API_KEY	ні	без неї листи не йдуть

Змінна	Обов'язкова	Призначення
RESEND_FROM	ні	відправник на верифікованому домені
ORDER_EMAIL_BCC	ні	прихована копія менеджеру

✅ **Правило.** `.env.example` у `git` із коментарями до кожної змінної, реальні значення — тільки у `.env` на сервері (`gitignore`). У нас кожен блок `.env.example` містить пояснення, де взяти значення й що станеться, якщо його не задати — це найдешевша документація з можливих.

14.3 Процедура оновлення

```
# локально
git commit && git push origin main

# на сервері
ssh <server>
cd /home/<user>/<project>
git pull
docker compose up -d --build
```

⚠️ **Граблі №1: базовий шлях білду.** Дефолт у `vite.config.js` — `/<repo>/` (для GitHub Pages). Прод мусить збиратися з `VITE_BASE_URL=/`:

```
"build:prod": "node scripts/gen-sitemap.cjs && VITE_BASE_URL=/ vite build"
```

Інакше ассети підуть на `/<repo>/...` і **React просто не стартує** — біла сторінка без помилок у консолі, окрім 404 на бандл.

⚠️ **Граблі №2: у репозиторії є workflow GitHub Pages** — він збирає **статичне дзеркало**, а не прод. Push у `main` не оновлює живий сайт. Це джерело плутанини «я ж запустив, чому нічого не змінилось».

14.4 Бекап

```
cp data/termojet.db data/termojet.db.bak-$(date +%Y%m%d) # перед кожною правкою даних
```

Оскільки БД — один файл, повний бекап = `scp` цього файлу + архів `uploads/`.

✅ **Правило: бекап перед кожним прогоном скрипта, що змінює дані.** Ми робимо це навіть для ідемпотентних — коштує секунду.

15. Дані: імпорт, синхронізація, прайс

15.1 ⚠ Найважливіше: дані товарів живуть у трьох місцях

#	Місце	Роль	У git?
1	жива БД (<code>backend/data/termojet.db</code>)	те, що показує прод	ні (volume)
2	<code>backend/seed-products.json</code>	початковий сідинг порожньої БД	так
3	<code>src/data/products.js</code>	статичний фолбек (режим без API)	так

API перекриває статистику, тому на проді показується БД. Але скрипт масової правки мусить оновлювати **всі три**, інакше: - правка лише в БД → зникне при переінсталяції з seed; - правка лише в seed → на проді не з'явиться (таблиця не порожня); - правка лише в статистиці → взагалі не побачить ніхто.

Саме тому наш `apply-seo-descriptions.js` оновлює всі місця, а `robust-apply.cjs` надійно патчить `products.js` за slug.

💡 **Зробити краще.** Це наш найгірший борг у роботі з даними. У новому проєкті: **одне джерело істини (БД)**, seed — лише для першого запуску, статичного фолбек-каталогу **немає взагалі**. Приберіть режим «сайт без API» — він дає подвійну ціну обслуговування.

⚠ **Граблі з Docker-контейнером.** В образ входять лише `backend/` і `dist/` — **немає** ні `src/`, ні `scripts/`, ні `_zvity/`. Тому файли даних для скриптів, які виконуються в контейнері, мусять лежати в `backend/scripts/`, інакше скрипт падає з `ENOENT`. Разові скрипти закидаються так:

```
docker compose cp scripts/seo-fix-titles.cjs app:/tmp/fix.cjs
docker compose exec app node /tmp/fix.cjs /app/backend/data/termojet.db --apply
```

15.2 Артикул як ключ звірки з обліковою системою

Артикули на сайті збігаються з 1С (звірено окремим звітом), тому **sku** — **ключ зіставлення** для прайсу, фідів і обліку.

Реальні складності, які виявила звірка (корисно знати заздалегідь):

- **суфікси значущі:** `84622480` («зі змішувачем») ≠ `84622480A` («з приводом»). Розширювати збіг по суфіксу заборонено;
- **префікс складу:** `TJ-EC-162` ↔ `84040TJ-EC-162` — потрібне зіставлення з префіксом;
- **код моделі шукається лише з межею слова**, інакше `НГ-48` чіпляється до `НГ-48A`;
- **варіанти виконання** (в ізоляції / без ізоляції) мусять мати різні артикули — ми додали суфікс `L` там, де код був спільний;
- **хвіст `L` у полі артикула** (`84030720 L`) призводив до дублювання позиції;
- **нечіткий збіг за назвою** застосовується **лише** до позицій без артикула.

✅ **Правило.** Артикул — це первинний ключ товару для бізнесу. Заводьте правила нумерації (варіанти, виконання, ліве/праве) **до** заповнення каталогу, і валідуйте унікальність при збереженні.

15.3 Прайс і фіди як похідні від каталогу

Генератор прайсу (`price/generate.py`) бере номенклатуру з **живого** `/api/products` і збирає три файли:

Файл	Для кого
XLSX (21 аркуш-розділ + «Зміст» із логотипом, фото товарів, посилання на сторінки)	клієнти, менеджери
CSV (UTF-8 BOM, роздільник <code>;</code>)	партнери, імпорт у CMS
XML-фід (RSS 2.0 + <code>g:</code>)	Merchant, маркетплейси

Опційно підмішуються позиції, яких ще немає на сайті (`--extra tovary.json`) — з локальними фото й перекладами; такі позиції **не потрапляють** у XML-фід (немає сторінки) і пропускаються, коли з'являються на сайті (звірка за slug).

 **Правило.** Прайс — **похідний артефакт**, а не окреме джерело даних. Редагувати XLSX руками не можна: наступна генерація перезапíše правки. Усі виправлення вносяться у **функції-нормалізатори генератора** (у нас це `fix_sku()` із 17 задокументованими правками) — тоді вони застосовуються повторювано.

15.4 Синхронізація наявності — не автоматизована

Наявність (`in_stock`) у нас **ставиться руками** в адмінці. Обмінник із обліковою системою (щоденний XML із залишками) заплановано, але не реалізовано.

 **Зробити краще для нового магазину.** Це те, що варто зробити на старті: щоденний імпорт залишків і цін з обліку → оновлення `price` , `in_stock` . Головна умова: імпорт **не має права** торкатися `seo_title` , `meta_description` , `description` та `i18n` (розділ 6.8).

16. Чеклісти

16.1 Запуск нового магазину — порядок дій

Етап 1. Каркас даних 1. Спроекувати таблицю товарів з обов'язковими `slug`, `sku`, `seo_title`, `meta_description`, `specs`, `is_visible`, `in_stock`, `currency`. 2. Довідник категорій — в БД або спільному JSON (не три копії в коді). 3. Правила нумерації артикулів (варіанти, виконання, ліве/праве). 4. Словник дозволених ключів характеристик по категорії.

Етап 2. Адмінка 5. CRUD товару з SEO-вкладкою (лічильники символів, прев'ю сніпета). 6. Мультизавантаження фото, керування галереєю. 7. Вкладки перекладів (якщо мов більше однієї). 8. Масові операції: CSV-імпорт/експорт, зміна цін по категорії. 9. Роль-модель і журнал дій.

Етап 3. Контент 10. Ресерч-бриф по кожній категорії (термінологія, ключі, FAQ, матриця сумісності). 11. Пілот: 1 товар → узгодження → 10 товарів → узгодження формату → масштаб. 12. Описи 300–450 слів за структурою розділу 7.2; короткі для аксесуарів. 13. `seo_title` (≤60) і `meta_description` (120–160) на кожну позицію. 14. Валідація: цифри зі specs, лінки реальні, брендів конкурентів немає. 15. Описи категорій: видимий текст + мета 120–155. 16. FAQ із реальних питань менеджерів. 17. Legal-сторінки: `privacy`, `terms`, `returns`.

Етап 4. SEO-технічка 18. Метадані в сирому HTML (SSR або ін'єкт) — перевірено `curl`. 19. `self-canonical`, `og:url` = `canonical`, один тег на сторінку. 20. JSON-LD: `Organization`, `Product+Offer` (лише з ціною), `Breadcrumb`, `FAQPage`, `Article`. 21. Білий список роутів → справжні 404. 22. `sitemap` (динамічний) + `robots.txt`. 23. `hreflang`, якщо мов більше однієї. 24. 301-карта зі старих URL, якщо є попередній сайт.

Етап 5. Комерція 25. Кошик, узгоджені назви полів (`quantity`!). 26. Серверний перерахунок суми. 27. Валюти й курс — один модуль. 28. Доставка (проксі перевізника з ключем на бекенді). 29. Форми: замовлення, консультація, партнерство.

Етап 6. Інтеграції й аналітика 30. GTM + GA4 ecommerce-події (з `ecommerce: null`). 31. UTM-захоплення + збереження в заявках. 32. Сповіщення (Telegram/e-mail), CRM, лист клієнту — усі `fire-and-forget`. 33. Фіди Merchant по мовах.

Етап 7. Інфраструктура 34. Docker + volumes для БД і завантажень. 35. nginx + TLS з автопродовженням; порт контейнера лише на localhost. 36. ENV-реєстр + `.env.example` з коментарями. 37. Бекап БД і завантажень (перевірити відновлення, а не лише створення). 38. Базовий аудит (`squirreldscan/Lighthouse`) → зафіксувати baseline.

16.2 Чекліст «додати товар» (для контент-менеджера)

- ☐ Назва — з моделлю й розрізняльним атрибутом
- ☐ Артикул за правилами нумерації, унікальний
- ☐ Категорія і підкатегорія
- ☐ Ціна + валюта (`0` = по запиту); акційна — якщо є
- ☐ Головне фото + галерея
- ☐ Характеристики типовими для категорії ключами
- ☐ Опис 300–450 слів за структурою; усі цифри — з характеристик
- ☐ 2–4 внутрішні лінки на сумісні товари/категорію
- ☐ `seo_title` ≤ 60, унікальний, з брендом

- ☐ `meta_description` 120–160
- ☐ Наявність (`in_stock`) виставлено
- ☐ «Показувати на сайті» увімкнено **після** перевірки решти
- ☐ Документація прив'язана (якщо є інструкція)
- ☐ Перевірено сторінку живцем: ціна, характеристики, лінки, вигляд у сніпеті

16.3 Чекліст перед деплоєм

- ☐ Білд зелений із **правильним** базовим шляхом
 - ☐ sitemap регенерований і закомічений (якщо не динамічний)
 - ☐ Бекап БД зроблений
 - ☐ Скрипти правки даних ідемпотентні й прогнані dry-run
 - ☐ Перевірено `curl` головної, категорії, товару: title/canonical/description/JSON-LD
 - ☐ Неіснуючий URL віддає 404
 - ☐ Форма замовлення проходить наскрізь (БД + сповіщення + лист)
 - ☐ Мобільна версія: тап по категорії з першого разу, кошик, форма
 - ☐ Після імпорту товарів — **прогнати нормалізацію SEO-полів**
-

17. Зведений реєстр граблів

#	Граблі	Наслідок	Профілактика
1	Дані товарів у трьох місцях	правка «не доїжджає» на прод	одне джерело істини
2	Ре-імпорт стирає <code>seo_title</code>	повернулись дублі title	імпорт не чіпає непорожні SEO-поля
3	Правки в БД у volume, не в git	втрата при переінсталяції	ідемпотентні скрипти в git
4	Довідник категорій у трьох копіях	текст для людей ≠ для Google	категорії в БД/спільному JSON
5	Конвертер валют у чотирьох копіях	різна ціна на сторінці/схемі/фіді	один модуль або ціна в грн у БД
6	<code>quantity</code> vs <code>qty</code>	«x 1» у сповіщеннях, хибна сума	єдиний словник полів API
7	EUR без конвертації в схемі	ціна в ~52x нижча для Google	конвертація в кожному виводі ціни
8	<code>trust proxy</code> не налаштований	rate-limit глобальний, вебхук 403 на всьому	ставити до лімітерів, перевірити живцем
9	<code>express.static</code> з <code>index: true</code>	головна без ін'єкту й hreflang	<code>index: false</code> , <code>redirect: false</code>
10	Ін'єкт по дефолтних рядках	тихо ламається при правці шаблону	явні маркери + smoke-тест
11	Регулярка canonical без <code>/></code>	«стрей <code>/></code> » у тексті, з'їдений hreflang	заміняти тег цілком
12	<code>react-helmet</code> + серверні теги	два <code><title></code> , дублі description	клієнт лише <code>document.title</code>
13	Prerender із мутацією <code><head></code>	hydration mismatch #418	серверний ін'єкт або справжній SSR
14	Нативні метадані React 19 у prerender	runtime-only, у знімку немає	не покладатися на DOM-snapshot
15	Чорний список у catch-all	soft-404, індексація порожніх сторінок	білий список роутів
16	Службові файли → SPA-заглушка	HTML замість 404, зламана валідація	404 текстом для <code>.xml/.txt/.json</code>
17	Стрип мовного префікса без перевірки	чинні <code>/en/...</code> отримували 301	301 лише якщо шлях є в карті
18	Голі <code><Link></code> замість <code>LLink</code>	вихід із мовної версії	лінтер-правило
19	Одне мовне дерево роутів	<code>/pl</code> → білий екран	генерувати дерево на кожну мову
20	<code>specs_<lang></code> з перекладеними ключами	позиційне зіставлення значень	машинні ключі, перекладати підписи
21	Медіа з <code>no-cache</code>	блмання фото, зайвий трафік	7 днів для медіа, рік для хешованих
22	Статичний каталог 2 МБ у бандлі	initial JS 3.6 МБ	динамічний <code>import()</code>
23	SVG у whitelist завантажень	stored XSS	без SVG
24	Секрети в <code>settings</code>	публічний GET віддає би токен	лише ENV

#	Граблі	Наслідок	Профілактика
25	Порт <code>0.0.0.0</code> у compose	обхід nginx/TLS	<code>127.0.0.1:8080:3000</code>
26	rate-limit приймача CRM 5/хв	втрата лідів на сплеску	звірити ліміт, черга з повторами
27	<code>Product</code> без <code>offers</code>	5 невалідних схем	без ціни → лише Breadcrumb+Organization
28	Числові слаги	нечитабельні URL, борг на 301	описові слаги з першого дня
29	Немає <code>width</code> / <code>height</code> у зображень	CLS	атрибути обов'язкові
30	Hover-стани на тач-екрані	категорія відкривається з другого тапу	гейтинг <code>canHover()</code> , тест на телефоні
31	<code>VITE_BASE_URL</code> дефолт для GH Pages	біла сторінка на проді	<code>VITE_BASE_URL=/</code> у <code>build:prod</code>
32	Workflow GH Pages ≠ прод	«запустив, а нічого не змінилось»	явно задокументувати
33	Таблиця характеристик у тілі опису	дубль канонічної секції	вирізати при рендері
34	Сумісність товарів «на око»	перегенерація 44 описів	матриця сумісності до генерації
35	Розмір гайки замість підключення	покупець бере не ті фітинги	документувати фактичний параметр
36	<code>migrateCurrency</code> перезаписує з seed	правка валюти в БД відкочується	seed не перезаписує наявні рядки
37	Оптимізація фото прив'язана до теки <code>/wp-content/</code>	нові фото в <code>/images/</code> без webp: 5 файлів >1 МБ, вага сайту 63.9 МБ, Images 100→86	стискати й конвертувати при завантаженні в API, ліміт ваги
38	Дедуп title стерто вдруге (11 сер)	5 груп дублів на 10 сторінках повернулись	захист у коді імпорту, а не в інструкції
39	Sitemap не регенеровано після заливки	12 нових товарів поза sitemap; 270 URL не закраулено	динамічний sitemap із БД
40	SSR-блок не для всіх типів сторінок	<code>/catalog</code> = 23 слова в сирому HTML, thin content на 82 стор.	наповнювати <code>#seo-content</code> для кожного типу сторінки
41	Бренд у кожному пункті SEO-списку	keyword stuffing: «termojet» 39.6 % на <code>/en</code>	бренд один раз у вступі блоку
42	Невалідний HTML в описі товару	<code></code> поза <code></code> , <code><tr></code> поза <code><table></code> → 9 a11y-помилки	валідація HTML при збереженні товару
43	EN <code>meta_description</code> не обрізали	170–182 символи на EN-товарах (ліміт 160)	обрізання для EN так само, як для <code>seo_title</code>
44	Закороткі описи статичних сторінок	<code>/service</code> 72, <code>/contacts</code> 99, <code>/about</code> 119 символів	мінімум 120 символів, перевіряти в CI

Додаток А. Приклад JSON-LD сторінки товару

```
[
  {
    "@context": "https://schema.org",
    "@type": "Product",
    "name": "Насосна група НГ-46 з термостатичним краном 1\"",
    "description": "Насосна група з термостатичним краном НГ-46 – попередньо зібраний вузол...",
    "sku": "84620460",
    "mpn": "84620460",
    "image": ["https://site/img/ng46-1.jpg", "https://site/img/ng46-2.jpg"],
    "brand": { "@type": "Brand", "name": "Termojet" },
    "category": "Насосні групи",
    "offers": {
      "@type": "Offer",
      "price": "7945",
      "priceCurrency": "UAH",
      "availability": "https://schema.org/InStock",
      "url": "https://site/catalog/nasosni-hrupy/ng-46p-z-termokranom-1",
      "seller": { "@type": "Organization", "name": "Termojet" }
    }
  },
  {
    "@context": "https://schema.org",
    "@type": "BreadcrumbList",
    "itemListElement": [
      { "@type": "ListItem", "position": 1, "name": "Головна", "item": "https://site" },
      { "@type": "ListItem", "position": 2, "name": "Каталог", "item": "https://site/catalog" },
      { "@type": "ListItem", "position": 3, "name": "Насосні групи", "item": "https://site/catalog/nasosni-hrupy" },
      { "@type": "ListItem", "position": 4, "name": "Насосна група НГ-46...", "item": "https://site/catalog/..." }
    ]
  },
  {
    "@context": "https://schema.org",
    "@type": "Organization",
    "name": "Termojet",
    "url": "https://site",
    "logo": "https://site/logo.svg",
    "description": "Виробник насосних груп, колекторів, клапанів та систем для котелень...",
    "foundingDate": "2002",
    "address": { "@type": "PostalAddress", "addressCountry": "UA", "addressLocality": "Київ" },
    "contactPoint": {
```

```
"@type": "ContactPoint", "telephone": "+380-XX-XXX-XX-XX",  
"contactType": "customer service",  
"availableLanguage": ["Ukrainian", "English", "Polish"]  
}  
}  
]
```

Додаток В. Шаблон ТЗ копірайтеру

Товар: {назва} · **Артикул:** {sku} · **Категорія:** {категорія} **Характеристики (єдине джерело цифр):** {повний JSON specs} **Сумісні товари (лінкувати):** {slug + анкор} **Категорія каталогу (лінкувати):** /catalog/{cat} **Основний ключ:** {ключ} · **Другорядні:** {2–3} · **Long-tail:** {2–3}

Завдання. Написати: - **seo_title** — 50–60 символів, з моделлю, закінчення « | Бренд»; - **meta_description** — 140–160 символів, вигода + факт + бренд; - **h1** — назва товару; - **description_html** — 300–450 слів, HTML (`<p>` , `<ul>` , `<li>` , `<strong>` , `<a>`), за структурою: вступ (40–60 сл.) → принцип роботи (60–90) → застосування (50–70) → переваги булетами (50–70) → технічні параметри (40–60) → міні-FAQ (2–3 питання); - **keywords** — 6–10; - **internal_links** — 2–4 об'єкти {**anchor, url**} **лише на реальні slug.**

Критично: 1. Кожна цифра — тільки з наведених specs. Немає у specs — не пишемо. 2. Бренди конкурентів не називати; лише «заміна імпортованому», «той самий клас». 3. Основний ключ — у першому реченні, далі 2–3 рази природно. 4. Тон інженерний, без маркетингової води. 5. Вихід — валідний JSON.

Додаток С. Команди щоденної експлуатації

```
# — Деплой —————
git push origin main
ssh <server> 'cd /home/<user>/<project> && git pull && docker compose up -d --build'

# — Бекап перед правками даних —————
cp data/<db>.db data/<db>.db.bak-$(date +%Y%m%d)

# — Скрипт правки даних у контейнері (dry-run → apply) —————
docker compose cp scripts/<script>.cjs app:/tmp/s.cjs
docker compose exec app node /tmp/s.cjs /app/backend/data/<db>.db
docker compose exec app node /tmp/s.cjs /app/backend/data/<db>.db --apply

# — Sitemap (якщо не динамічний) —————
node scripts/gen-sitemap.cjs && git add public/sitemap.xml

# — Перевірка сирого HTML очима краулера —————
curl -s https://<domain>/catalog/<cat>/<slug> | grep -E
'<title>|canonical|hreflang|og:url|ld\+json'
curl -s -o /dev/null -w '%{http_code}\n' https://<domain>/неіснуючий-url # → 404

# — Логи вебхуків —————
docker logs -f <container> | grep <webhook>

# — Регенерація webp —————
find <uploads> -type f \( -iname '*.jpg' -o -iname '*.png' \) ! -name '*.webp' \
| while read f; do [ -f "$f.webp" ] || cwebp -quiet -q 82 "$f" -o "$f.webp"; done
```

Підсумок: сім рішень, які дали найбільше

- 1. `seo_title` і `meta_description` — окремі поля БД, заповнені на 100 % позицій. Один цей крок прибрав 269 задовгих title і 361 дубль description.
- 2. **Серверний ін'єкт метаданих і контенту в сирий HTML**. Оцінка 73 → 81, Core SEO 90 → 99, і сайт став видимим для LLM-ботів, які не виконують JS.
- 3. **Описи 300–450 слів із реальними цифрами й перелінковкою** — 331 позиція за конвеєром з обов'язковою валідацією.
- 4. **Білий список роутів замість чорного** — справжні 404 замість індексації порожніх сторінок.
- 5. **Fire-and-forget для всіх зовнішніх сервісів** — жодна інтеграція не може зламати оформлення замовлення.
- 6. **Ідемпотентні міграції й скрипти даних** — перезапуск контейнера самолікує дані, повторний прогон скрипта безпечний.
- 7. **Серверний перерахунок суми замовлення** — ціни клієнта не існує для бекенду.

І сім речей, які в новому магазині варто зробити інакше:

- 1. Фреймворк із SSR замість ручного ін'єкту в CSR SPA.
- 2. Одне джерело істини для товарів (без seed-копії та статичного фолбеку).
- 3. Категорії в БД, а не три копії в коді.
- 4. SEO-поля, галерея й переклади — **в адмінці**, а не в скриптах.
- 5. Один модуль курсу валют (або ціни в гривні прямо в БД).
- 6. Машинні ключі характеристик + словник дозволених ключів по категорії.
- 7. Автоматичний імпорт цін і залишків з обліку — з гарантією, що він **не чіпає** SEO-поля й описи.

Найголовніше, що показав аудит 17 серпня

Перші шість пунктів вище — про **правильний стан**. Регресія 81 → 77 за п'ять тижнів довела, що цього замало: правильний стан не тримається сам.

Усе, що зашите в код, вистояло — структуровані дані, Social, Mobile, i18n, Local SEO, Analytics лишились на 100, Core SEO на 94. **Усе, що трималось на дисципліні людини, просіло**: вага фото, довжина описів, валідність HTML, повторний прогін нормалізації title, актуальність sitemap.

Тому восьме — і найважливіше — рішення для нового магазину: **зробити неправильний стан технічно неможливим**.

Ризик	Захист, який не можна забути виконати
важкі фото	стиснення й webp/avif при завантаженні в API + ліміт ваги
стерті SEO-поля	імпорт не перезаписує непорожні <code>seo_title</code> / <code>meta_description</code>
дублі title	нормалізація вбудована в імпорт, не окремий скрипт
застарілий sitemap	динамічний sitemap із БД
биті теги в описі	валідація HTML при збереженні товару

Ризик	Захист, який не можна забути виконати
задовгі/закороткі мета	перевірка довжини у формі товару + у CI
тиха регресія	аудит у CI : падіння оцінки блокує реліз

Останній рядок вартий окремої уваги: якби прогін аудиту стояв у пайплайні, ми побачили б ці 4 бали 11 серпня — у день заливки, а не через тиждень.